UAS OATH Token Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAS OATH Token Implementation Guide..... 3
2. Introduction to UAS OATH Token Implementation..... 4
3. UAS OATH Soft Token Activation Process..... 9
  3.1. YESsafe Token+ 4..... 10
  3.2. V-Key OATH..... 15
  3.3. Other Authenticator Apps..... 18
4. UAS OATH FM with HSM..... 20
  4.1. UAS OATH FM with HSM Configuration..... 26
5. UAS OATH Configuration..... 40
6. UAS OATH Token Implementation Using API..... 54
  6.1. Token Activation..... 55
  6.2. Login..... 61
  6.3. Token Verification..... 68
  6.4. Token Management..... 73
  6.5. Errors..... 81
7. UAS OATH Token Housekeeping Policy..... 82
8. Appendices..... 83

1. Preface - UAS OATH Token Implementation Guide

This document provides detailed information on the integration and configuration of OATH-based tokens with the AccessMatrix Universal Authentication Server (UAS). In addition, it consists of a detailed API integration guide for OATH-based tokens with the AccessMatrix UAS system.

Intended Reader

This is intended as a reference for the security administrators, i-Sprint's Professional Service personnel, and developers.

Document Scope

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix Common Administration Guide*
- *UAS Administration Guide*
- *AccessMatrix REST API Specification*

2. Introduction to UAS OATH Token Implementation

AccessMatrix UAS system supports the integration of OATH-based tokens to address the strong authentication required for a wide range of enterprise applications. The solution provides administration, authentication, authorization, and audit services to address security and compliance requirements.

Key Benefits

- **Enhanced security with Multi-Factor Authentication**
  Multi-factor authentication provides exceptionally high levels of security to protect against hacking attacks. Two-factor authentication (a type of multi-factor authentication) with OATH-based tokens is used to secure network access, protect financial accounts, digitally sign transactions, and keep criminals from causing losses by preventing unauthorized network access.
- **Standardized to allow better interoperability**
  OATH-based tokens are following standardized specifications (RFC6283, RFC6287 and RFC6030). This means, various token vendors may create their own tokens that follow the OATH specifications. The OATH tokens can work with UAS to provide authentication. Note that activation of software-based token (such as mobile token) is not standardized and may still require UAS to support the activation process specifically.
- **Scalable**
  You can add more users, and use the tokens to secure more and other business applications.

UAS Key Features

- **Multi-Factor Authentication**
  This adds additional security measures to the standard username, password logins across the AccessMatrix UAS system that stops unauthorized logins, even the passwords that have been shared among different services.
- **Token Management**
  The administrator can determine and manage the user-to-token relationships.
- **Token Grouping Management**
  The administrator can configure the token batches and groups to the respective users.
- **Token Integration APIs**
  OATH token offers APIs integration to the AccessMatrix UAS system.

- **Ready Integration with Leading HSM Vendors**
  Integration is easy to carry out with various HSM vendors.
- **Security Auditing and Reporting**
  Audit reports are available in a wide selection of PDF, HTML and XML formats.
- **Scalable and High Availability Architecture**
  It supports applications that can utilize industry-standard protocols and applications along with high-availability support.

Manage OATH Token Life Cycle

AccessMatrix supports multiple authentication methods to enable organizations to deploy the appropriate level of authentication technologies to address security requirements. The following section describes how you can manage the life cycle of OATH tokens.

- **Token Initialization (Hardware tokens and some software tokens)**
  Token vendor normally carries out the token initialization. No UAS component is involved in the token initialization process. Token initialization process typically involves provisioning a unique token seed into the hardware token and generating of the token seeds file (PSKC file) that are encrypted by a transport key. These token seeds will need to be provisioned/imported into UAS using the PSKC file. Some software/mobile tokens may have this process as well, but instead of provisioning the token to the hardware token, the token vendor may provision the token seed to its provisioning server that will be further provisioned to the mobile token during activation process.
- **Token Activation (software/mobile tokens)**
  For software/mobile tokens, the token seed needs to be provisioned into the mobile phone (specifically into the token app or the token component embedded in a mobile app). This process is known as activation process. There are a few ways to do token activations:
    - **YESsafe Token 4**
      For YESsafe Token 4, when the token app/component requests for activation, UAS generates the token seed at the server-side and provision it in encrypted form to the token app/component.
    - **V-Key OATH Token**
      For V-Key OATH Token, the token seeds are provisioned into the V-Key provisioning server. The PSKC files containing the token seeds and activation PINs are provisioned to UAS (via Token Import). During token activation, the activation PIN will be sent to the

token app (e.g. via QR or retrieved in the background) and the token app will download the V-Key V-OS token firmware containing the token seed into the token app/component.

    - **Google Authenticator**

      Google Authenticator way of OATH token provisioning has become a norm that multiple authenticator/token apps are supporting this type of token provisioning. UAS generates the OATH token seed. The token seed is formatted in Google Authenticator specification and displayed as a QR code to be scanned by the Google Authenticator app or other compatible app.

- **Token Administration**

  The system administrator sets up the token parameters, token groups and batches, and the administrators' rights.

    - **Token Import**

      Token import is a bulk job process which is also an asynchronous process that will run in the background. First, the token seeds are imported into the UAS repository using the Token Import Utility (TIU). Token Import Utility requires two administrators assigned with Token Import right to log in. Once the tokens are imported successfully, they will be in a re-synchronous state.

    - **Token Inventory**

      The assigned token administrator will take charge of the token inventory in the UAS repository.

Token Management

Manage different models of OATH tokens by carrying out the following:

- **Token Assignment**

  Assign an active token to a user by carrying out the token assignment in the Admin Console in UAS. The administrator can use either the System Assigned option or the token serial number to assign to the users. The system assigned method tells the system to assign an available token from a selected token group to the user. Use the token serial number to assign a token to the user(s) manually.

  > **Note**: Only the administrator with the assigned token right can assign a token to a user.

- **Token Un-Assignment and Status Change**

  The administrator can unassign the tokens from the users and change the token status in the Admin Console.

  > **Note**: Need to change the status of an un-assigned token to "active" before it can be re-assigned.

- **Token Re-Sync**

  Token may be out-of-sync with the security server due to the time drift from the GMT. To resolve this problem, the user's token needs to be resynchronized.

- **Replacement over time**

  The administrator can set the life span of a token to be replaced after a specific timeline period on the token status field.

Implementation Checklists

To use OATH tokens for multi-factor authentication on AccessMatrix UAS, you must first configure the token-related policy and objects. Depending on the token vendor, you may need to import the token seeds and/or activate the tokens.

> **Notes**:
> - For UAS to work with OATH tokens, there is no additional token vendor library required.
> - To carry out the token administration works, you can configure the global settings for the token management, and set up the Token Import Utility (TIU) before you configure the token policy.

Token(s)	Reference
Configure token policy	Configure OATH Token Policy
Configure token batch	Configure Token Batch
Configure token group	Configure Token Group
Import OATH token (hardware/V-Key OATH)	Import OATH Token
User(s)	**Reference**
Configure login module	Configure Login Module
Configure Authentication Realm	Configure Authentication Realm
Configure user accounts	Configure User Accounts

Token(s)	Reference
Assign token to user	Assign Token to User

Implementation Planning

During implementation, you need to understand the customer's nature of business, studying the customer's requirements and its existing environments to carry out the action plans. The following discuss the security and integration considerations.

Integration Consideration

Before the users can use the tokens, tokens must first be assigned to them. You set the relationships between the tokens and the users first before assigning the tokens to the users. The information (user-to-token relationships) of assignment can be stored either in UAS or in the application database, depending on the tokens used by one application or multiple applications.

- **User-to-token relationships (assignment/unassignment) are maintained by UAS**
  AccessMatrix Universal Authentication Server (UAS) maintains the token serial number and user Id mapping information. You do not keep this information including the token serial number in the application database if:
  - User-to-token serial number relationships are stored in UAS.
  - Multiple applications need to share the tokens, use UAS to maintain the user-to-token relationships.
- **User-to-token relationships are maintained by the application database**
  The application database maintains the token serial number and user Id mapping information. You do not keep this information including the token serial number in AccessMatrix UAS if:
  - User-to-token serial number relationships are stored in the application database.
  - Only one application needs to use the tokens, use the application database to maintain the user-to-token relationships.

3. UAS OATH Soft Token Activation Process

For OATH soft token, the activation process is varied from one vendor to another as the steps normally involve proprietary method in the protocol to provision the token seed to the soft token. UAS supports the activation process for the following OATH soft tokens:

- YESsafe Token+ 4
- V-Key OATH
- Google Authenticator (or Google Authenticator compatible token apps)

3.1. YESsafe Token+ 4

The YESsafe Token+ 4 can be activated through either online and offline methods. For details on how to configure the UAS Token Activation Portal (TAP) for online/offline YESsafe token activation, refer to *UAS TAP Deployment Guide - YESsafe Token Deployment*.

Online Activation

Online activation of the YESsafe token involves the generation of a token seed in runtime in UAS. The token seed is then provisioned to the mobile token in an encrypted format. The token seed is encrypted using a public key, generated by the YESsafe token, and an activation password, generated by UAS. The token seed is encrypted during transport (on top of SSL), with the activation password providing another layer of encryption.

There are a few ways to provision the activation password to the token:

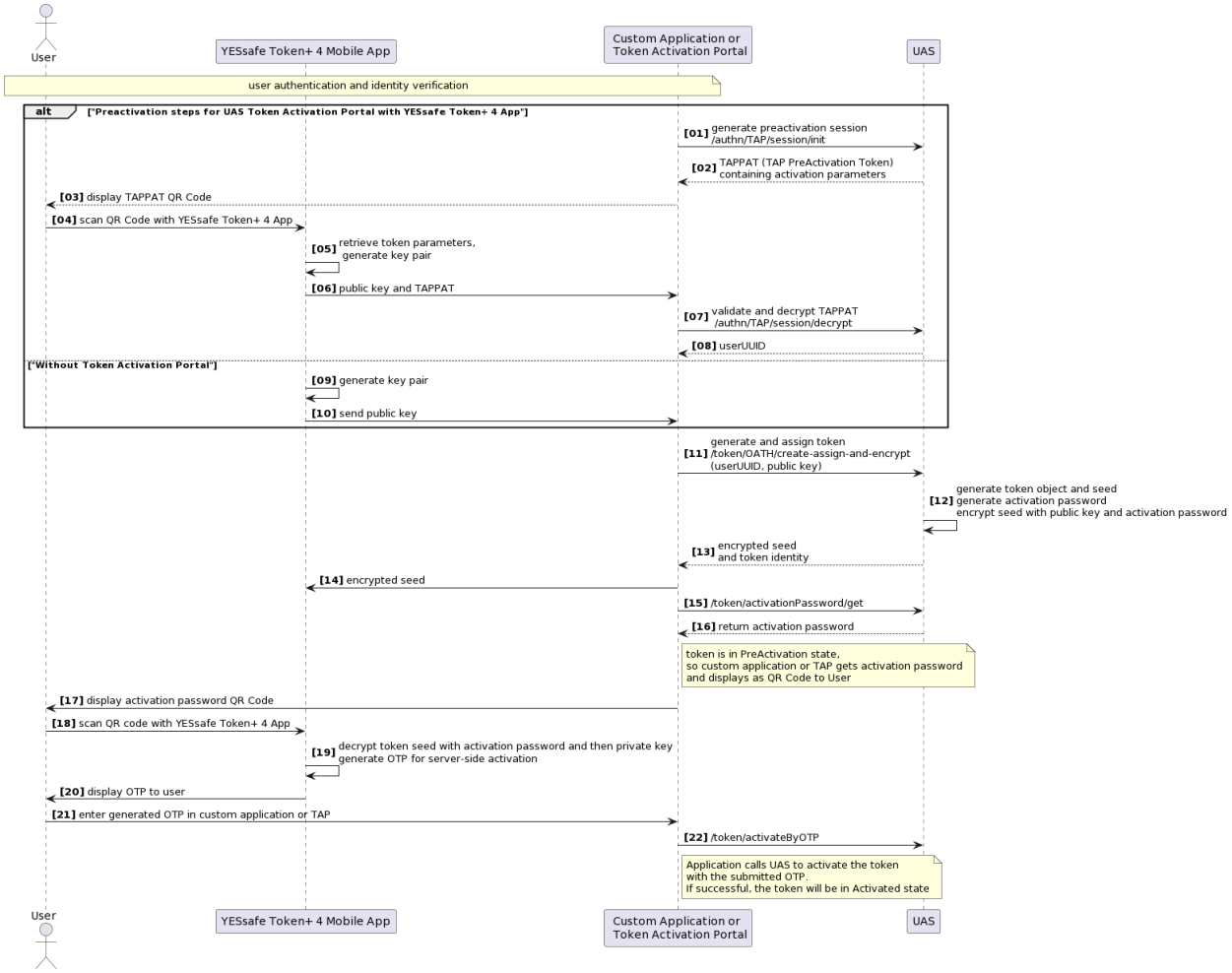
- It can be retrieved non-interactively (not known to the user) via application's API.
- It can also be displayed as QR code to be scanned by the token app.

> **Note**: The sequence diagram in the following section illustrates the scenario when the activation password is provisioned via QR code.

YESsafe Token+ 4 is also available as an SDK to be integrated/embedded into a mobile app (e.g. internet banking app), or as a standalone app available in Android Play Store and Apple AppStore.

UAS provides a Token Activation Portal that can work with the YESsafe Token+ 4 app (not SDK). This solution is meant to be utilized for quick deployment without customization. The YESsafe Token+ 4 SDK can be used for a more flexible or personalized experience. For this solution, the Token Activation Portal is not needed. Instead, the application has more control over the token activation experience.

Sequence Diagram



Sequence Diagram Steps Explanation for YESsafe Token+ 4 App and UAS Token Activation Portal (Online)

- User logs in and identifies themselves to UAS TAP
- UAS TAP calls UAS to generate a PreActivation Token, which contains the token parameters required to initialize the token
- UAS TAP displays the PreActivation Token as a QR code to user
- User scans the QR code with the YESsafe token which then:
    - Reads in token parameters
    - Generates a key pair
- YESsafe token sends the PreActivation Token and public key to UAS TAP

- UAS TAP asks UAS to validate and decrypt the PreActivation token, and the user identity is returned
- UAS TAP asks UAS to generate the token seed, create the token and assign it to the user; the token will be in PreActivation state
- Token seed is encrypted with the public key and activation password before it is sent it back to the YESsafe token
- UAS TAP obtains the activation password from UAS and displays it as QR code
- User uses the YESsafe token to scan the QR code to activate the token and get the activation password
- YESsafe token decrypts the token seed with the activation password and its private key, and generates an OTP for server-side activation
- User enters the OTP in UAS
- UAS TAP calls UAS with the OTP to activate the token; if successful, the token will be in the Activated state

Sequence Diagram Steps Explanation for YESsafe Token+ 4 SDK and Custom Application (Online)

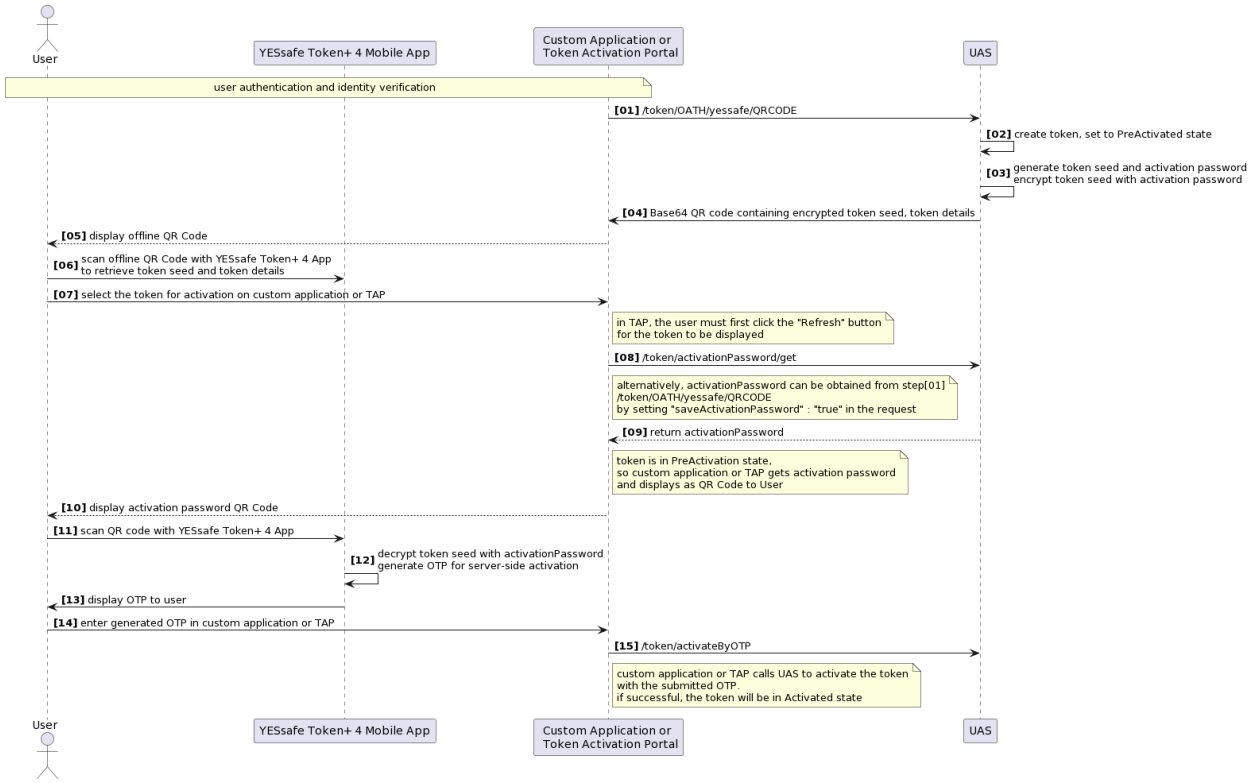
- User logs in and identifies themselves to the custom application
- User/custom application initiates the token activation process
- Token app with YESsafe Token+ 4 SDK generates a key pair
- Token app sends the public key to the custom application
- Custom application asks UAS to generate the token seed, create token and assign it to the user; the token will be in PreActivation state
- Token seed is encrypted with the public key and activation password before it is sent back to the token app
- Custom application obtains activation password from UAS and displays it as a QR code
- User uses the token app to scan the QR code to activate the token and get the activation password
- Token app decrypts the token seed with the activation password and its private key
- Token app generates an OTP for server-side activation
- User provides the OTP to the custom application

- Custom application calls UAS with the OTP to activate the token; if successful, the token will be in the Activated state

Offline Activation

Offline activation of the YESsafe token is also available for use cases where the user does not have internet access. As the user is considered offline, the pre-activation step is skipped. UAS generates both the token seed and activation password, and encrypts the token seed with the activation password. The Initializing Vector (IV), encrypted token seed, and other token blob details such as the token suite and algorithm, are returned and can be displayed as a QR code to be scanned by the application.

Sequence Diagram



Sequence Diagram Steps Explanation for YESsafe Token+ 4 App and UAS Token Activation Portal (Offline)

- User logs in and identifies themselves to UAS TAP
- UAS TAP asks UAS to generate the token seed and encrypts it with the activation password; the token will be in the PreActivation state

- The encrypted token seed, as well as token details such as algorithm and token suite, are returned and displayed as a QR code on UAS TAP
- User uses the YESsafe token to scan the QR code and retrieve the encrypted token seed and token details
- User clicks the **Refresh** button on UAS TAP, then selects the token to be activated and clicks **Next**
- UAS TAP obtains the activation password from UAS and displays it as a QR code
-

> **Note**: The activationPassword can also be retrieved at the start of the flow when calling /token/OATH/yessafe/QRCODE by setting the saveActivationPassword parameter to "true" in the request. This can be used in a custom application implementation of the YESsafe Token+ 4 App token activation flow.

- User uses the YESsafe token to scan the QR code to retrieve the activation password
- YESsafe token decrypts the token seed with the activation password and generates an OTP for server-side activation
- User enters the OTP in UAS TAP
- UAS TAP calls UAS with the OTP to activate the token; if successful, the token will be in the Activated state

3.2. V-Key OATH

For details on how to configure the UAS Token Activation Portal (TAP) for V-Key token activation, refer to *UAS TAP Deployment Guide - V-Key Token Deployment*.

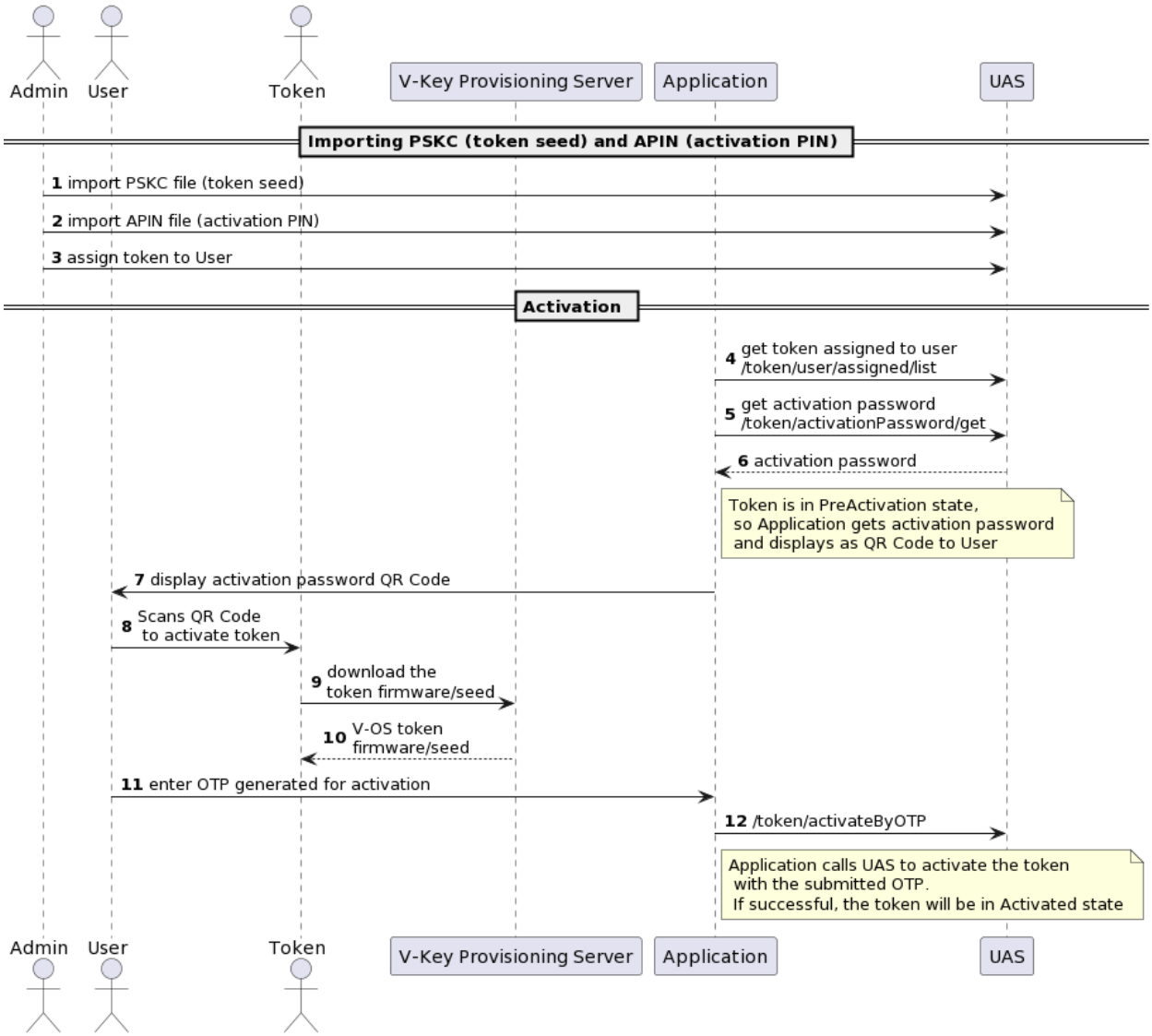
V-Key OATH has two PSKC files: token seed PSKC and activation PIN PSKC. Both need to be imported using Token Import Utility (TIU). This is to provision the token seed to UAS and also to store the activation PIN in UAS. The activation PIN will be used during the activation process.

V-Key soft token is normally embedded inside the mobile app itself (e.g. the internet bank mobile app). The activation process involves the activation PIN and serial number to be provided to the soft token. This can be done via QR code, or more commonly in practice, the mobile app acquires the activation PIN and serial number from its server-side. The application server-side calls UAS REST API to get the token serial number and the activation PIN. The mobile app will then retrieve a token firmware (V-OS) containing the token seed based on the token serial number from V-Key Provisioning Server. V-Key secures this process and the token seed by providing device binding, checking firmware integrity and using V-OS secure storage.

For V-Key Cloud, the customer, or the application owner, should first have a registered V-Key Cloud account

- V-Key will then generate token seeds and activation pins in the form of PSKC and APIN xml files
- These files will be used for importing into UAS

Sequence Diagram



Sequence Diagram Steps Explanation

- Customer uses AccessMatrix Admin Console to import the PSKC and APIN xml files into UAS
    - As a result, V-Key tokens will be created in UAS; tokens will be in PreActivation state
    - The activation process is started by assigning an imported token to a user
- User installs the V-Key Token application into her mobile device
- User logs in and is identified to application
- Application calls UAS to get activation password and generates a QR code for it
- User scans the QR code to activate the token; the token will then download the token seed from V-Key Provisioning Server

- An OTP is then generated for token activation in UAS
- Application forwards the OTP to UAS for verification; if successful token will be in Activated state

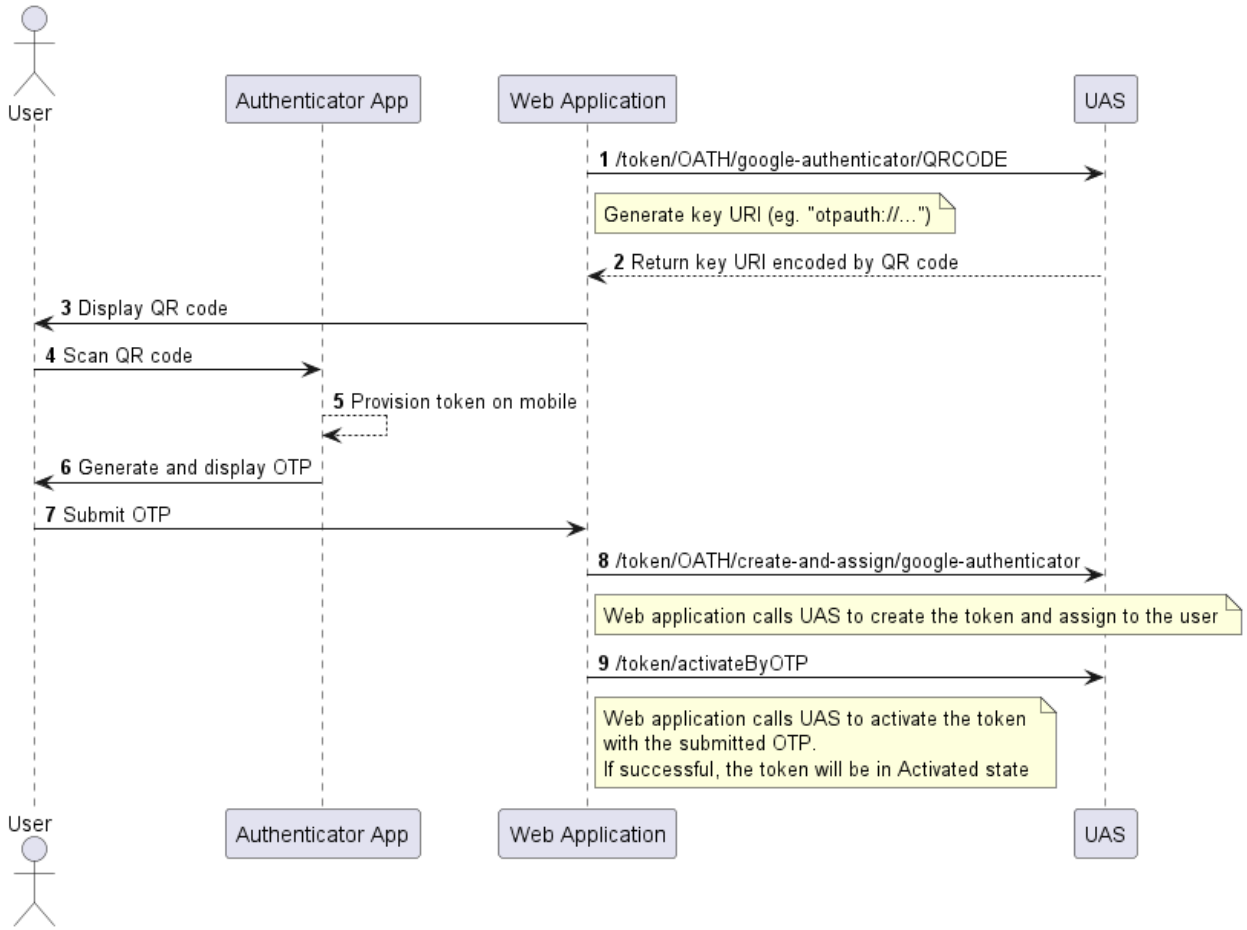
3.3. Other Authenticator Apps

This section expands on the OATH soft token activation process for other authenticator apps that are commonly used. These includes apps such as Google Authenticator and Microsoft Authenticator.

> **Note:** For more details on how to configure the UAS Token Activation Portal (TAP) for activating these authenticator apps, please refer to:
>
> - *UAS Token Activation Portal Deployment Guide - Google Authenticator Token Deployment*
> - *UAS Token Activation Portal Deployment Guide - Microsoft Authenticator Token Deployment*

The activation for these authenticator apps involves generating token seed during runtime in UAS, and provisioning the token seed to the authenticator app via QR codes.

Sequence Diagram



Sequence Diagram Steps Explanation

- User has logged in and identified themselves to the application.
- Application calls UAS to generate a Key-URI Format string containing OATH token seed.
- UAS encodes key-URI into a QR code and returns it to the application.
- Application displays the QR code to the user.
- User scans QR code with their choice of authenticator app.
- Authenticator app provisions the new token.
- Authenticator app generates and displays the OTP to the user.
- User submits the OTP to the application.
- Application calls UAS to create and assign a server side OATH token to the user.
- Application calls UAS to verify the submitted OTP. If successful, the token will be in 'Activated' state.

4. UAS OATH FM with HSM

> **Note**: Skip this section if you do not intend to use OATH-based services with HSM(s).

This page provides information on the concept and implementation of the AccessMatrix Universal Authentication Server (UAS) OATH Functionality Module (FM) with various Hardware Security Modules (HSMs). The OATH Functionality Module (FM) is a designed, security-sensitive program code application which can be uploaded into the HSM and operate as part of the HSM firmware to provide OATH-based services.

AccessMatrix Universal Authentication Server (UAS) supports the integration of the OATH Functionality Module (FM) with:

- SafeNet ProtectServer HSM (and SafeNet Jcprov Failover)
- Utimaco CryptoServer HSM

The solution provides integration, administration, and signing and authentication services using the OATH FM with HSMs to protect cryptographic keys that contain sensitive data against compromise and attacks. See the required components below for the implementation:

- **Hardware**
    - Supported Hardware Security Modules (HSMs)
    - AccessMatrix UAS server(s)
- **Software**
    - AccessMatrix UAS application
    - Drivers of supported HSMs

> **Notes**: Always use the latest version of the drivers bundled with the HSM.
> - Supported environment: Oracle JRE/JDK 1.8 (minimum is 1.8.0_181_b13).
> - Supported platforms: Windows, Linux and Solaris. This guide and related guides focus mainly on Windows and Linux platforms.

Implementation Checklist

To implement the UAS OATH FM with supported HSMs, ensure the following tasks are completed (in addition to those detailed within this guide):

- SafeNet ProtectServer HSM

- Utimaco CryptoServer HSM

SafeNet ProtectServer HSM

Task	Reference	Guide
Install Oracle JDK/JRE	Install Oracle JDK/JRE	*UAS E2EEA Developer Guide - UAS E2EEA with HSM Implementation - UAS E2EEA with SafeNet ProtectServer HSM Implementation - Deployment*
Install SafeNet ProtectServer HSM Drivers	Install SafeNet ProtectServer HSM Drivers	
Initialize HSM Adapter	Initialize Adapter	
Initialize Token Slot	Initialize Token	
Deploy OATH FM to SafeNet ProtectServer HSM	Deploy OATH FM to SafeNet ProtectServer HSM	
Create HSM Keys	Create HSM Keys	*UAS OATH Token Implementation Guide - UAS OATH FM with HSM Configuration*
Secure AccessMatrix Server License	Upload HSM OATH License	
Configure OATH Token Policy	Configure HSM OATH Token Policy	
Configure the OATH Login Module	Configure Login Module	*UAS OATH Token Implementation Guide - UAS OATH Token Configuration*
Create the Authentication Realm	Configure Authentication Realm	
Assign Authentication Realm to User	Configure User Accounts	
Configure OATH Token Batch	Configure Token Batch	
Configure OATH Token Group	Configure Token Group	
Import HSM OATH Token via PSKC File	Import OATH Token	
Assign HSM OATH Token to User	Assign Token to User	

Utimaco CryptoServer HSM

Task	Reference	Guide
Install Oracle JDK/JRE	Install Oracle JDK/JRE	*UAS E2EEA Developer Guide - UAS E2EEA with HSM Implementation - UAS E2EEA with*
Install CryptoServer Host Software	Install CryptoServer Host Software	
Set Up Utimaco CryptoServer LAN Device	Set Up Utimaco CryptoServer LAN Device	

Task	Reference	Guide
Enable Utimaco CryptoServer Administration Using Smart Card or Keyfile	Enable Utimaco CryptoServer Administration Using Smart Card or Keyfile	*Utimaco CryptoServer HSM Implementation - Deployment*
Configure PKCS#11	Enable PKCS#11	
Deploy E2EEA FM to Utimaco CryptoServer HSM	Deploy OATH FM to Utimaco CryptoServer HSM	
Create HSM Keys	Create HSM Keys	*UAS OATH Token Implementation Guide - UAS OATH FM with HSM Configuration*
Secure AccessMatrix Server License	Upload HSM OATH License	
Configure OATH Token Policy	Configure HSM OATH Token Policy	
Configure the OATH Login Module	Configure Login Module	*UAS OATH Token Implementation Guide - UAS OATH Token Configuration*
Create the Authentication Realm	Configure Authentication Realm	
Assign Authentication Realm to User	Configure User Accounts	
Configure OATH Token Batch	Configure Token Batch	
Configure OATH Token Group	Configure Token Group	
Import HSM OATH Token via PSKC File	Import OATH Token	
Assign HSM OATH Token to User	Assign Token to User	

Implementation Planning

Planning prior to environment setup is critical when implementing the UAS OATH FM with HSMs. Understanding the customer's nature of business, studying the customer's requirements and its existing environments will help to identify the actions to take prior to the actual implementation.

The following sections discuss the implementation considerations that should be assessed during planning.

Security Considerations

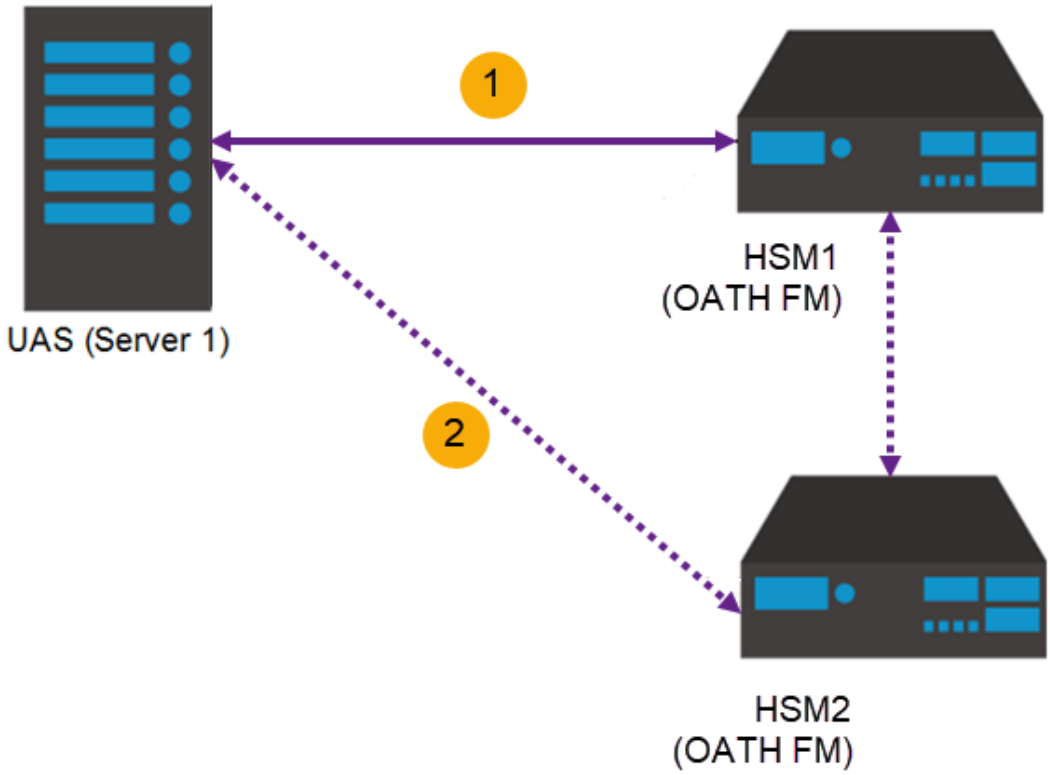
HSMs securely manage and stores digital keys used during transactions, for identification, and applications access. Check the security services your HSM provides when you carry out this implementation.

- Upon successfully carrying out the integration of OATH FM with the HSM, the data is typically protected based on the application, with **OATH FM** handling the data.
- By using the **HSM Database Storage Key,** the data can be of any size and typically protected for any authorized entities on AccessMatrix UAS server(s).

  > **Note**: If you have a second UAS server, the same database can be connected from the first UAS server to the second UAS server.

- When defining the token policies, it is necessary to make them intuitive for the security officer and administrator to carry out their jobs.
- Together with the customer, it is essential to provide security services by defining the **key strengths' standards** and **relationships** to meet the organization's requirements. Refer to the section UAS OATH FM with HSM Configuration to specify the relationships, and using which key types and mechanisms, including its class and category when creating the secret keys.

For the architecture design, it is recommended to use a minimum of **two HSM devices with appropriate backups** when integrating with OATH FM. Once an HSM is enabled, the operation cannot be undone since it is a one-way, irreversible process. If the first HSM fails, AccessMatrix UAS will connect to the second HSM.



Integration Considerations

You must first install and configure the supported HSM device based on the guide for the relevant HSM in *UAS E2EEA Developer Guide - UAS E2EEA Implementation with HSM*. Complete all of the necessary steps detailed in the *Deployment* portion of the guide before installing the AccessMatrix UAS application server and carrying out the integration of the OATH FM with the SafeNet ProtectServer HSM. This includes the following steps:

- Install the HSM software in the host computer system.
- Install the HSM package that includes the device driver(s) and confirm the correct operation of the adapter and driver installation.
- Ensure the driver(s) are running correctly.
- Install the HSM's Application Programming Interface (API) or net server software supplied along with the product, if any.
- Test the HSM drivers and library installed; ensure token is configured, key pair and certificate are generated.

- Deploy the OATH FM to the HSM.

> **Note**: In the respective implementation guide of the supported HSM, the OATH FM may be referred to as the E2EEA FM. These names are used interchangeably, as both the OATH and E2EEA functionalities are implemented in one FM, i.e. the E2EEA FM. In this guide, the term OATH FM is used instead of E2EEA FM.

Performance Considerations

It is necessary to enforce the key management roles to allow a high level of accountability for the performance of key management operations. A well-defined set of users, administrators and security officers ensure that adequate operational controls can be enforced throughout the key management lifecycle. For example, impose multi-person control over critical operations. There must have relevant controls over the policy assets for different types of cryptography and keys used in the data protection applications.

4.1. UAS OATH FM with HSM Configuration

This section details the configuration steps relating to the implementation of the UAS OATH FM with HSM.

Setup Steps

This section defines the steps relating to the implementation of the UAS OATH FM with supported HSMs. Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concept of the UAS OATH FM with HSM solution.

- **Create HSM keys**
  A hardware security module (HSM) is a physical computing device that safeguards and manages digital keys for strong authentication and provides cryptoprocessing. Refer to Create HSM Keys to know the required keys for this UAS OATH FM solution.
- **Upload HSM OATH License**
  Server license is used to enable general and product specific features. For HSM OATH license, refer to Upload HSM OATH License for details.
- **Configure the HSM OATH Token Policy**
  HSM OATH token policy is used for V-Key and other standard OATH tokens such as i-Sprint's OATH token. If this policy is used, then OATH token generation and encryption will be done in the HSM. Refer to Configure HSM OATH Token Policy for details.
- **Other Configuration Items**
  Token group, token batch, and OATH login module and its authentication realm, need to be configured and created via Admin Console.
  Refer to UAS OATH Token Configuration for the rest of the configuration items in Admin Console.

Create HSM Keys

The following keys are required to be generated for the OATH FM with HSM solution:

- **Token Storage Key**
  The key used to encrypt the token seed when given to the AccessMatrix server by the HSM.

- **Key Encryption Key (KEK)**

  The key used to wrap/unwrap the encrypted transport key.

Generate Token Storage Key

Perform the following steps to generate the Token Storage Key to the OATH token slot initialized in the *Deployment* section:

SafeNet ProtectServer HSM

1. Launch **Key Management Utility** by running KMU (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\KMU.bat*).
2. Select the previously initialized token and enter the User PIN.
3. Click **Options > Create > Secret Key** menu. The **Generate Secret Key** dialog box is displayed.
4. Select "AES" as the **Mechanism**.
5. Enter a meaningful storage key name (**Label** value), e.g. "StorageKey".
6. Enter the **Key Size (bits)** value, e.g. "256".
7. Specify what attributes a key will have by checking/unchecking the key attribute checkboxes.

   > **Note**: The table in the following sub-section describes the attributes that can be set when creating a key using the Key Management Utility (KMU). For different customers, there will be different expectations for their keys. The list below is customizable according to the requirements of different customers.

8. Click **OK** to confirm the generation of secret key.
9. The generated key will be displayed in the **Objects on Selected Token** box on the main KMU interface.

Meaning of Key Attributes

Key Attribute	Description
Persistent	Stores the object on non-volatile memory. Persistent objects can be accessed after session termination. Selected by default, and cannot be deselected (unclickable).
Extractable	An extractable key can be wrapped (encrypted with another key) and extracted from the HSM.
Derive	Indicates that the key can be used in key derivation functions.
Export	Indicates the key may be used to export other keys (similar to the wrap function).
Verify	Indicates that the key may be used for verifying signatures or MAC values.
Sensitive	If a key is sensitive, the key's value may not be revealed in plain text. Once a key becomes sensitive it cannot be modified to be non-sensitive.
Exportable	An exportable key may be wrapped (encrypted with another key), but only with keys marked with the Export attribute.
Encrypt	Indicates that the key may be used for encryption.

<table>
<tr><td></td><td>
<table>
<tr><th>Key Attribute</th><th>Description</th></tr>
<tr><td>Wrap</td><td>Indicates that the key may be used to wrap other keys.</td></tr>
<tr><td>UnWrap</td><td>Indicates that the key may be used to unwrap keys.</td></tr>
<tr><td>Modifiable</td><td>Indicates whether or not the object is modifiable, that is, if the object's attributes may be modified after creation.</td></tr>
<tr><td>Private</td><td>Defines whether the object is protected by the user PIN. A private object is only accessible to an application that has supplied the user PIN.</td></tr>
<tr><td>Sign</td><td>Indicates that the key may be used for signing.</td></tr>
<tr><td>Decrypt</td><td>Indicates that the key may be used for decryption.</td></tr>
<tr><td>Import</td><td>Indicates the key may be used to import other keys.</td></tr>
</table>
</td></tr>
<tr><td>Utimaco CryptoServer HSM</td><td>
1. Launch the PKCS#11 CAT software.
2. Select the relevant slot under Slot List.
3. Click Login/Logout in the toolbar.
4. Select "Login User" and log in.
5. Click Object Management* in the toolbar. This brings up the Object Management page.
6. Click Generate > Generate Key.
7. Click Create Attribute List.
8. Select "CKA_Label" from the drop-down list and click Add.
9. Enter the name of the key, e.g. "StorageKey".
10. Click on the OK button.
11. Click Generate.
</td></tr>
</table>

Generate Key Encryption Key (KEK)

Key Encryption Key (KEK) is a special key created with the wrap attribute, allowing it to be used for wrapping other KEKs and/or data keys. KEKs are usually created as split custodian keys because of their enhanced security.

<table>
<tr><td>SafeNet ProtectServer HSM</td><td>
Important Note: Prior to KEK generation, the following steps must be performed to enable Key Component creation.
1. Launch Safenet, INc. -- Adapter Management by running the gCTAdmin file (e.g. C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\gCTAdmin.bat).
2. Enter User PIN and proceed to login.
</td></tr>
</table>

3. After successful login, select **Edit > Security Mode** menu. The **Manage Tokens** dialog box is displayed.
4. Select **Custom** radio button to display the **Custom Security Flags** dialog box.
5. Select the **Weak PKCS#11 Mechanisms** checkbox and click the **OK** button to close the dialog box.
6. Click the **OK** button in the **Modify Security Mode** dialog box.
   A restart alert message will be displayed.
7. Click **OK** to restart CtAdmin.
8. Proceed to create the key components.
   The key component can be created by either using **Generate Key Components** or **Enter Key from Components** function. Refer below for the steps.

> **Note**: After successful generation of the key components, repeat the same steps above but deselect the **Weak PKCS#11 Mechanisms** checkbox as setting this flag compromises security.

Generate Key Components:

This step will create a random key as a number of components. These components may be recorded manually so that it can be entered on another machine by using the **Enter Key from Components** function. Generating a key component makes it possible to create a key whose value is unknown to any single party. Only by combining the components known by each custodian can the key be regenerated.

> **Note**: Only perform the following steps if the Key Components are not generated yet.

To generate key components:

1. Launch **Key Management Utility** by running **KMU.bat** (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\KMU.bat*).
2. Select the previously initialized token (e.g. "OATHKEYS") and enter the **User PIN**.
3. Click **Options > Create > Generate Key Components** in the menu. The **Create Key Components** dialog box is displayed.
4. Select "AES" as the **Mechanism**.
5. Enter a label for the key into the **Label** field, e.g. "KEK".
6. Enter the **Key Size (bits)** value, e.g. "256".
7. Click **OK** to continue.
8. When prompted by the KMU, enter in the **Number of Components** field the number of components that you wish the key to be split into, e.g. "2".

   > **Note**: There is no limit on the number of components.
9. Click **OK** to start displaying the key components. A **Ready to generate component** dialog box will be displayed for each component determined in step 8.
10. Record the **Component Value** and **Key Check Value (KCV)**, both given in hexadecimal, displayed in these dialogs. The KCV for the generated component is

used to verify correct entry of the component during manual key component entry.

> **Note**: The generated key will be displayed in the **Objects on Selected Token** box on the main KMU interface.

Entering a Key from Components:

In a Portable Symmetric Key Container (PSKC) file, the transport key (i.e. Session Encryption Key (SEK)) is encrypted by KEK which is a key from HSM. KEK should be loaded into HSM in Parts/Key Components. Regenerate the KEK from the component values and follow the steps below to create KEK.

> **Note**: Only perform the following steps if the Key Components are not generated yet.

To generate a key from components:

1. Launch **Key Management Utility** by running KMU (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\KMU.bat*).
2. Select the previously initialized token (e.g. "OATHKEYS") and enter the **User PIN**.
3. Click **Options > Create > Enter Key From Components** in the menu. The **Generate Secret Key** dialog box is displayed.
4. Select "AES" as the **Mechanism**.
5. Enter a label for the key into the **Label** field, e.g. "KEK".
6. Enter the **Key Size (bits)** value, e.g. "256".
7. Specify what attributes a key will have by checking/unchecking the key attribute checkboxes.
8. Click **OK** to continue.
9. When prompted by the KMU, enter the number of key components to combine in the Number of Components field, e.g. "2".

   > **Note**: There is no limit on the number of components.
10. Click **OK** to continue. An **Information Message** dialog box will be displayed.
11. Click **OK** to open the **Ready to accept component** dialog box. A number of component dialogs will appear, corresponding with the number of components specified.

> **Notes**:
>
> - The KCV appears automatically when the key component is entered, allowing the custodian to confirm correct entry. The KMU will check that the KCV matches that of the key components being entered. If a mismatch is detected, an error is shown.
> - After entering the key components, the regenerated key will be displayed in the **Objects on Selected Token** box on the main KMU interface.

Utimaco CryptoServer HSM	Secure the appropriate Key Encryption Key from your token vendor.

Upload HSM OATH License

Before performing any configurations in the Admin Console, secure the server license required to implement the OATH FM with the supported HSM. Ensure that the "HSMOATH" vendor is added in the **Token Vendors** license feature to enable OATH FM support.

To load the server license:

1. Navigate to **Administration Dashboard > System > Server & Cache** and click **Server** on the left pane.
2. Select the **Server Id** and click **Edit**.
3. Select the **Licensed Features** tab and select the first radio button. Browse for the license file.
4. Click **Upload Licence File** to upload the license file to the server.
5. Save the changes.
6. Restart the AccessMatrix server for the new license to take effect.

Configure HSM OATH Token Policy

To configure HSM OATH token policy, refer to the respective sections:

- SafeNet ProtectServer HSM
- Utimaco CryptoServer HSM

SafeNet ProtectServer HSM

To create a new HSM OATH token policy for SafeNet ProtectServer HSM, navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane. Click **Create** button and select the "HSM (SafeNet ProtectServer) OATH Token Policy" implementation from the **Choose Token Policy Implementation** page. Then, click **Next** and provide the details for the login policy attributes before saving the changes.

Attribute Name	Description
*Token Policy Id	Unique token policy Id, e.g "HSMOATHTokenPolicy".
*Token Policy Name	Name of token policy, e.g. "HSMOATHTokenPolicy".
Description	Description of token policy. (Optional)
Version	Token policy entry version. (Uneditable)
Implementation	Type of token policy implementation, i.e. "HSM (SafeNet ProtectServer) OATH Token Policy". (Uneditable)
Encryption key	Encryption key for this token policy, e.g. "AEK1".
Attributes Tab	
*HSM User PIN	This is the same PIN you used to login to Key Management Utility (KMU) or HSM Slot.
Resynchronize Event Window	When the token time is out-of-sync, resync mode can be activated at the AccessMatrix server to enlarge the event window for the next verification. This setting is used to determine the acceptance event window when it is in resync mode. Range of values: 5 to 2000. Default: 300
Resynchronize Time Window	When the token time is out-of-sync, resync mode can be activated at the AccessMatrix server to enlarge the time window for the next verification. This setting is used to determine the acceptance time window when it is in resync mode. Range of values: 5 to 2000. Default: 200
Event Window	Determines the verification window of the counter difference between Token and AccessMatrix server. This is used for verification of the event/counter-based token. If the OTP generated by the token is within the server's acceptance time window, then the verification is considered successful. Range of values: 2 to 200. Default: 10
Time Window	Determines the verification window of the time difference between Token and AccessMatrix server. This is used for verification of time-based token. If the OTP generated by the token is within the server's acceptance time window, then the verification is considered successful. Range of values: 2 to 200. Default: 10

Attribute Name	Description
Signature Window	Range of values: 2 to 200. Default: 10
Allow Multiple Signing In The Same Time Step	If set to "true", multiple transaction signing and verification in the same time step is allowed. If set to "false", multiple transaction signing and verification at the same time step will be considered as a replay attack. Default: false
Enable Time Drift	If set to "true", OTP verification allows time drift in the token (suitable for hardware token). If set to "false", OTP verification assumes there is no time drift in the token. Default: true
Lock Token If Bad Verification Count Is More Than	Specifies the number of bad verification attempts to be exceeded before locking the token. If specified as "0", any number of bad verification attempts will not lock the token. Range of values: 0 to 100. Default: 3
Challenge Format Length	The length of the challenge for challenge-response authentication. Range of values: 4 to 10. Default: 6
Default Suite of Time-Based Token	This value would be used if suite value is not specified during token import for time-based token. Possible values: • "HmacSHA1" • "HmacSHA256" • "HmacSHA384" • "HmacSHA512"
Default Suite of Event-Based Token	This value would be used if suite value is not specified during token import for event-based token. Possible values: • "HmacSHA1" • "HmacSHA256" • "HmacSHA384" • "HmacSHA512"
Preferred application ID for OTP (TOTP) if more than one blob	The default application ID of Time-based One-Time Password (TOTP) for a token that has more than one TOTP token blobs. This will be used if there is no specific application ID specified in the API call.
Application ID Naming Strategy	Select the option for naming the Application ID for each type of application(TOTP, HOTP, OCRA):

Attribute Name	Description
	• If "System Assign Running Number" is selected, a running number will be appended to the Algorithm and set as the Application ID, e.g. TOTP1, TOTP2, TOTP3... • If "Extract Application ID from concatenated KeyID" is selected, the Application ID will be extracted from the originally concatenated 'SerialNumber+AppId' of the OATH PSKC keyId. Only usable on OATH Tokens with KeyID that is prefixed with the token serial number ("SerialNumber+AppId"). Default: System Assign Running Number
PIN/Activation Password Import Mode	Select the import mode for PIN/Activation Password. • For encrypted PIN/Activation Password, select "Encrypted PIN Import" mode. • For plain value PIN/Activation Password, select "As-Is Import" mode. • For PIN/Activation Password encrypted with HSM encrypted transport key, select "PIN Import with HSM Encrypted Key". Default: Encrypted PIN Import
Generate & Send Mobile Activation QRCode After Assignment	This feature currently supports V-Key Mobile Tokens only. If set to "true", the server will generate Mobile Activation QRCode upon successful token assignment. This Activation QRCode will be sent to the user through an event notification policy. Select the corresponding event notification policy in **Event Notification Policy for Activation QRCode** attribute. Default: false
Event Notification Policy for Mobile Activation QRCode	Specifies the event notification policy to be used for sending the Mobile Activation QRCode.
Deferred Signature Window	The window for the deferred signature that is generated and meant to be verified later; this window is much larger than the default signature window. Default: 0 (deferred signature is not enabled)
Transport Key Label	The key in the HSM used to encrypt the token blob during import process.
Transport Key Wrap Key Label	The key in the HSM used to wrap/unwrap the transport key, which is used to encrypt the token blob during import process, e.g. "KEK".
*Encryption Key Label	The key used to encrypt the token seed when given to the AccessMatrix server by the HSM, e.g. "StorageKey".
*HSM Slot Label	Specifies the slot or token labels for each HSM, e.g. "OATHKEYS".

Utimaco CryptoServer HSM

To create a new HSM OATH token policy for Utimaco CryptoServer HSM, navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane. Click **Create** button and select the "HSM (Utimaco CryptoServer) OATH Token Policy" implementation from the **Choose Token Policy Implementation** page. Then, click **Next** and provide the details for the login policy attributes before saving the changes.

Attribute Name	Description
*Token Policy Id	Unique token policy Id, e.g "HSMOATHTokenPolicy".
*Token Policy Name	Name of token policy, e.g. "HSMOATHTokenPolicy".
Description	Description of token policy. (Optional)
Implementation	Type of token policy implementation, i.e. "HSM (Utimaco CryptoServer) OATH Token Policy" (Uneditable)
Attributes Tab	
*HSM User PIN	This is the same PIN you used to log in to the HSM Slot.
HSM Slot ID	The slot containing your keys for this module. Default: 0
*Device	The CryptoServer HSM device address. Example: "3001@192.168.1.49", where "192.168.1.49" is the IP address and "3001" is the port of CryptoServer LAN address.
Connection timeout (seconds)	Specifies the maximum time in seconds to wait before the connection establishment is aborted if the device is not responding. Default: 3
Command Timeout (seconds)	Specifies the maximum time in seconds to wait for the answer from CryptoServer after sending a command. Default: 60
Maximum Connections Total	Specifies the maximum number of connections in the pool. Default: 2
*Encryption Key Label	The key used to encrypt the token seed when given to the AccessMatrix server by the HSM, e.g. "StorageKey".
Transport Key Label (importing token blob	The key in the HSM used to encrypt the token blob during import process.

Attribute Name	Description
encrypted by this key inside HSM)	
Transport Key Wrap Key Label (importing token blob encrypted by a transport key that is itself encrypted by this key inside HSM)	The key in the HSM used to wrap/unwrap the transport key, which is used to encrypt the token blob during the import process, e.g. "KEK".
Resynchronize Event Window	When the token time is out-of-sync, resync mode can be activated at the AccessMatrix server to enlarge the event window for the next verification. This setting is used to determine the acceptance event window when it is in resync mode. Range of values: 5 to 2000. Default: 300
Resynchronize Time Window	When the token time is out-of-sync, resync mode can be activated at the AccessMatrix server to enlarge the time window for the next verification. This setting is used to determine the acceptance time window when it is in resync mode. Range of values: 5 to 2000. Default: 200
Event Window	Determines the verification window of the counter difference between Token and AccessMatrix server. This is used for verification of the event/counter-based token. If the OTP generated by the token is within the server's acceptance time window, then the verification is considered successful. Range of values: 2 to 200. Default: 10
Time Window	Determines the verification window of the time difference between Token and AccessMatrix server. This is used for verification of time-based token. If the OTP generated by the token is within the server's acceptance time window, then the verification is considered successful. Range of values: 2 to 200. Default: 10
Signature Window	Range of values: 2 to 200. Default: 10
Allow Multiple Signing In The Same Time Step	If set to "true", multiple transaction signing and verification in the same time step is allowed. If set to "false", multiple transaction signing and verification at the same time step will be considered as a replay attack. Default: false

Attribute Name	Description
Enable Time Drift	If set to "true", OTP verification allows time drift in the token (suitable for hardware token). If set to "false", OTP verification assumes there is no time drift in the token. Default: true
Lock Token If Bad Verification Count Is More Than	Specifies the number of bad verification attempts to be exceeded before locking the token. If specified as "0", any number of bad verification attempts will not lock the token. Range of values: 0 to 100. Default: 3
Challenge Format Length	The length of the challenge for challenge-response authentication. Range of values: 4 to 10. Default: 6
Default Suite of Time-Based Token	This value is used if suite value is not specified during token import for time-based token. Possible values: • "HmacSHA1" • "HmacSHA256" • "HmacSHA384" • "HmacSHA512"
Default Suite of Event-Based Token	This value is used if suite value is not specified during token import for event-based token. Possible values: • "HmacSHA1" • "HmacSHA256" • "HmacSHA384" • "HmacSHA512"
Preferred application ID for OTP (TOTP) if more than one blob	The default application ID of Time-based One-Time Password (TOTP) for a token that has more than one TOTP token blobs. This will be used if there is no specific application ID specified in the API call.
Application ID Naming Strategy	Select the option for naming the Application ID for each type of application (TOTP, HOTP, OCRA): • If "System Assign Running Number" is selected, a running number will be appended to the Algorithm and set as the Application ID, e.g. TOTP1, TOTP2, TOTP3... • If "Extract Application ID from concatenated KeyID" is selected, the Application ID will be extracted from the originally concatenated "SerialNumber+AppId" of the OATH PSKC keyId. Only usable on OATH Tokens with KeyID that is prefixed with the token serial number ("SerialNumber+AppId"). Default: System Assign Running Number

Attribute Name	Description
PIN/Activation Password Import Mode	Select the import mode for PIN/Activation Password. • For encrypted PIN/Activation Password, select "Encrypted PIN Import" mode. • For plain value PIN/Activation Password, select "As-Is Import" mode. • For PIN/Activation Password encrypted with HSM encrypted transport key, select "PIN Import with HSM Encrypted Key". Default: Encrypted PIN Import
Generate & Send Mobile Activation QRCode After Assignment	This feature currently supports V-Key Mobile Tokens only. If set to "true", the server will generate Mobile Activation QRCode upon successful token assignment. This Activation QRCode will be sent to the user through an event notification policy. Select the corresponding event notification policy in the **Event Notification Policy for Activation QRCode** attribute. Default: false
Event Notification Policy for Mobile Activation QRCode	Specifies the event notification policy to be used for sending the Mobile Activation QRCode.
Deferred Signature Window	The window for the deferred signature that is generated and meant to be verified later; this window is much larger than the default signature window. Default: 0 (deferred signature is not enabled)

5. UAS OATH Configuration

Setup Steps

This section defines the steps relating to the configuration of the UAS OATH Token solution.

- **Configure the global setting for token management**
  The administrator can use the **Token Management** tab in **Global Setting** to modify the Token Import Utility (TIU) features and token functions for various OATH token models. To access this page, in the Admin Console, navigate to **Administration Dashboard > Configuration > Global Setting** and click the **Token Management** tab. Modify the token management attributes such as **Token Status After Unassign**, **Token Status (if threshold exceeded)**, and other hardware token-related attributes based on the necessary requirements. Refer to *UAS Administration Guide - Token Management* for additional details.
- **Setup Token Import Utility (TIU) Configuration**
  Token Import Utility (TIU) configuration is where the administrator defines the list of token groups available for selection during the token import process, whether or not overwriting of an existing token is allowed, as well as the maximum number of tokens for token group reassignment. To configure this, navigate to **Administration Dashboard > Configuration > Token > TIU Configuration** and click the **Edit** button. Modify the attributes based on the necessary requirements. Refer to *UAS Administration Guide - TIU Configuration* for more details.
- **Configure the OATH token policy**
  OATH token policy manages the behavior of the OATH token used for authentication. A predefined OATH token policy called "DefaultOATHTokenPolicy" is created by default. However, a new token policy can also be created. If multiple OATH token vendors are to be used in UAS, it is highly recommended to create separate token policies for each of the different OATH token vendors. For example, if both YESsafe tokens and V-Key OATH tokens are used, at least two token policies should be created such that YESsafe tokens should use the token policy meant for YESsafe and V-Key OATH tokens should use another token policy meant solely for V-Key OATH tokens. Refer to Configure OATH Token Policy for policy details.
- **Configure the token batch**
  Token batch refers to the physical grouping of token devices. This is normally used for hardware tokens. For example, this may be used to group tokens by batch of delivery from the token vendor. For software/mobile tokens, this is normally not used. However, all tokens, including software tokens, must belong to a token batch. Creation of the token batch can be done in the

Admin Console or during token import using the Token Import Utility (TIU). Each batch is tied to a particular token policy. For OATH token implementation, if multiple OATH token vendors are required, each OATH token vendor should use a separate token policy; likewise, each vendor should use a separate token batch. Refer to Configure Token Batch for configuration details.

- **Configure the token group**

  Token group refers to the logical grouping of OATH token devices. Token group is used for token management. For example, UAS contains a feature to manage system token assignment based on token groups. The creation of a token group can be done in the Admin Console or during token import using TIU. Refer to *UAS Administration Guide - Token Group* for more details.

- **Import OATH token (hardware token and V-Key OATH token)**

  For hardware tokens and V-Key OATH tokens, before a token can be used, it must first be imported to the AccessMatrix system via the Token Import Utility (TIU), an application used for bulk import of tokens into the database. Only PSKC files are supported for the token import. A PSKC file contains token information which includes the token seed, OATH authentication mechanism or OATH suite, and token serial number information. Refer to Import OATH Token for the import steps.

- **Configure Login Module**

  OATH token login is an OATH token-specific login module that provides an OATH token-based login method for users who may reside in either an internal or external user store. A predefined OATH token login module called "DefaultOATHTokenLogin" will be created upon installation. However, a new login module can also be created. Refer to Configure Login Module for the configuration steps.

- **Configure Authentication Realm**

  Authentication realm associates user credentials with authentication method(s) based on a lookup module and login module(s). User entries can be created in a local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). By default, a predefined OATH authentication realm called "DefaultOATHToken" will be created upon the successful installation of AccessMatrix. However, a new authentication realm can also be created. Refer to *AccessMatrix Common Administration Guide - Configure Authentication Realm* for configuration details.

- **Assign login accounts to the user**

  A user is an entity that accesses the system and may belong to a default store (local repository)

or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the **Login Accounts** tab. Refer to Configure User Accounts for details.

- **Assign OATH token to the user**

  The token can be assigned and unassigned in the Admin Console. This can either be done through system assignment or by manually entering the token serial number. Refer to Assign Token to User for details.

Configure OATH Token Policy

To configure the OATH token policy, navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane. Select the pre-created token policy: "DefaultOATHTokenPolicy" or click **Create** and select "OATH Token Policy" implementation from the **Choose Token Policy Implementation** page and click the **Next** button. Then, specify the details for the token policy attributes before saving the changes.

Attributes	Description
Attributes Tab	
Encryption Key	Token Encryption Key used to encrypt the token blob.
Resynchronize Event Window	The number of incremented counters to verify when the token is in resync mode. Resync is used when the token counter and AM server counter are out-of-sync. Range of values: 5 to 2000. Default: 300
Resynchronize Time Window	The number of acceptable timesteps for verification when the token is in resync mode. Resync is used when the token time and AM server time are out-of-sync. Range of values: 5 to 2000. Default: 300
Event Window	The maximum number of incremented counter that AM server uses for verification. This is used for HOTP blob. Range of values: 2 to 200. Default: 10
Time Window	The number of timesteps that determines the acceptable time difference between a token and AM server for verification. This is used for TOTP blob. This difference is adjusted to the last known shift for each verification if the **Enable Time Drift** attribute is set to "true". The time step is determined by the token during token import or

Attributes	Description
	creation. Range of values: 2 to 200. Default: 10
Signature Window	This is used for OCRA blob to determine the verification window. How it is used depends whether the blob is time-based or event-based. If time-based, this is the number of timesteps for the acceptable time difference between a token and AM server for verification. If event-based, this is the maximum number of incremented counter that AM server uses for verification. Range of values: 2 to 200. Default: 10
Allow Multiple Signing In The Same Time Step	If "true", multiple transaction signing and verification in the same time step is allowed. If "false", multiple transaction signing and verification in the same time step will be considered as replay attack. Default: false
Enable Time Drift	If enabled, OTP verification allows time drift in token (suitable for hardware token). If disabled, OTP verification assumes there is no time drift in token. Default: true
Challenge Format Length	The number of digits of the generated challenge when a challenge response is used. Default: 6
Lock Token If Bad Verification Count Is More Than	Specifies the number of bad verification attempts to be exceeded before locking the token. If specified as "0", any number of bad verification attempts will not lock the token. Range of values: 0 to 100 Default: 3
Default Suite of Time-Based Token	This value would be used if suite value is not specified during token import for time-based token.
Default Suite of Event-Based Token	This value would be used if suite value is not specified during token import for event-based token.
Preferred application ID for OTP (TOTP) if more than one blob	The application ID to be used If more than OTP (TOTP) blobs are available and application ID is not specified during verification.
Application ID Naming Strategy	Select the option for naming the Application ID for each type of application (TOTP, HOTP, OCRA): • If "System Assign Running Number" is selected, a running number will be appended to the Algorithm and set as the Application ID. e.g. TOTP1, TOTP2, TOTP3...

Attributes	Description
	• If "Extract Application ID from concatenated KeyID" is selected, the Application ID will be extracted from the originally concatenated "SerialNumber+AppId" of the OATH PSKC KeyID. Only usable on OATH Tokens with KeyID that is prefixed with the token serial number ("SerialNumber+AppId"). • If "Use KeyID as is" is selected, the KeyID will be used as the Application ID as it is without being changed. Default: System Assign Running Number
Deferred Signature Window	Window for deferred signature that is generated and meant to verified later; this windows is much larger than the default signature window. Default: 0 (deferred signature is not enabled)
Vendor Specific Tab	
Google Authenticator	
Issuer	The name of the service/issuer. This name will be shown in the Google Authenticator app. This is used to differentiate the token of one issuer from the token of another. Default: AM5
V-Key	
PIN/Activation Password Import Mode	Select the import mode for PIN/Activation Password. • For encrypted PIN/Activation Password, please select "Encrypted PIN Import" mode. • For plain value PIN/Activation Password, or if "As-Is import" is intended, please select "As-Is Import" mode. Default: Encrypted PIN Import
Generate & Send Mobile Activation QRCode After Assignment	This feature currently supports V-Key Mobile Tokens only. If enabled, server will generate Mobile Activation QRCode upon successful token assignment. This Activation QRCode will be sent to user through event notification policy. Please select corresponding event notification policy in **Event Notification Policy for Activation QRCode** attribute. Default: false
Event Notification Policy for Mobile Activation QRCode	Specify the event notification policy to be used for sending the Mobile Activation QRCode.

Configure Token Batch

To configure a token batch, navigate to **Operation Dashboard > Token > Token Batch** and click **Create** button and provide the details for the token batch attributes before saving the changes.

Attribute	Description
*Token Batch Id	Identification assigned to the token batch.
*Token Batch Name	The name of the token batch.
Description	Brief description for the token batch.
Attributes Tab	
*Token Policy	Specify the OATH token policy, e.g. "DefaultOATHTokenPolicy".
Static Vector	(Not applicable)
Manufacture Time	The date/time when the token is manufactured.
Expiry Time	The token expiry date/time.
Token Attributes Template Id	The identification of the template is used to set token attributes for additional custom information.

Configure Token Group

To configure a token group, navigate to **Operation Dashboard > Token > Token Group** and click **Create** button and provide the details for the token group attributes before saving the changes.

Attributes	Description
*Token Group Id	Identification assigned to the token group.
*Token Group Name	The name of the token group.
Description	Brief description of the token group.
*Token Vendor	The token vendor to manage in this group. Ensure that the correct token vendor (i.e. OATH) is configured to group similar tokens.
Segment	Name of the segment the token group belongs to.
Attributes Tab	

Attributes	Description
System Token Assignment	
The system will, by default, assign the oldest token first based on the time a token is placed in a token group. It is particularly useful for hardware tokens that have an expiry date due to the lifespan of the battery. For soft tokens, this is no longer required. It has a higher rate of token assignment as users change their devices such as mobile phones, etc. more often than hardware tokens. A soft token vendor such as V-Key sometimes requires the new tokens to be re-provisioned when they update their firmware. To allow higher system-token assignment capacity and to increase performance, it is highly recommended to set **Randomize Reservation Pool** to "true" and **Database Fetching Order of Reservation Pool** to "None". These settings fetch the token reservation pool from the database quicker and reduce the chance of conflict and clashing during the token assignment for a more efficient system-token-assignment process. If the system expects a more extensive concurrent system-token-assignment request, increase the value of **Reservation Pool Size.**	
Randomize Reservation Pool	This is to randomize the available tokens when assigning to a user if there are large concurrent requests for system token assignment . Select the setting "true" for soft tokens that may require token re-provisioning due to soft token SDK/firmware upgrade. It will fetch the token reservation pool from the database much quicker and reduce the chance of conflict and clashing during the token assignment. Default: false
Reservation Pool Size	Set the reservation pool size for random system token assignment. When handling high concurrent requests, set **Randomize Reservation Pool** to "true" and then set this pool size to at least 20. Default: 10
Database Fetching Order of Reservation Pool	For fetches from the database to get the reservation pool, this setting specifies the sorting of the fetch. Select the setting "None" for soft tokens that may require token re-provisioning due to soft token SDL/firmware upgrade, such as V-Key soft tokens. Setting the order to "None" will reduce the database processing tremendously. Default: With Ordering

Import OATH Token

Before a token can be assigned to a user, it needs to be imported to the AccessMatrix system first. The Token Import Utility (TIU) allows the administrator to import a token, delete a token, re-assign a token group, and perform various other token-related operations.

> **Note:** TIU will only display the buttons based on the admin rights granted to a user.

To launch the TIU:

1. Click **Cancel** to close the Admin Console login page.
2. Navigate to **Token > Token Import Utility**.

3. Enter the first token admin's login Id and password (e.g. tokenAdmin1) and click **OK**.
4. Enter the second token admin's login Id and password (e.g. tokenAdmin2) and click **OK**. The TIU page will be displayed after a successful login.

To import tokens:

1. Click **Import Token**.
2. Select an **Import Action** option.

> **Note:** By default, the "Token Seeds Import" option is selected. Select **Token Seeds Import** to perform a normal token import.

3. Specify the OATH PSKC file to be used for token import, by completing either of the following sub-steps (a,b):
    a. **Upload local file** - Perform the following steps:
        i. Click the corresponding radio button.
        ii. Click **Choose File** and select the OATH PSKC file to be uploaded.
        iii. Click **Upload**.
    b. **Or specify server file under system temp folder** - Perform the following steps:
        i. Click the corresponding radio button.
        ii. Place the OATH PSKC file in your AccessMatrix server's *temp* folder.

> **Note:** This is typically located at *<ACMATRIX_ROOT>/tomcat/temp/.*

        iii. Next, specify the name of the OATH PSKC file (e.g. "example.xml").

> **Note:** You do not need to specify the path to the temp folder.

4. Set up **Token Batch** by completing either of the following sub-steps (a,b):
    a. Select the OATH token batch from the drop-down list.

> **Note:** Refer to the steps described in Configure Token Batch section on how to create a token batch using the Admin Console.

    b. Create a new token batch via TIU, if necessary:
        i. Click **Create** to display the **Create Token Batch** dialog box.
        ii. Specify the ***Token Batch Id**, ***Token Batch Name**, and **Token Policy** values.

> **Note:** Select the token policy with "OATH Token Policy" implementation.

        iii. Click **Submit**.

5. Set up **Token Group** by completing either of the following sub-steps (a,b):

    a. Specify the appropriate OATH token group:

    > **Note:** Refer to the steps described in Configure Token Group section on how to create a token group using Admin Console.

        i. Click the drop-down list to display the **Select Token Group** dialog box.

        ii. Click **Search.**

        iii. Select the token group, and click **Add to Selection.**

        iv. Click **Apply.**

    b. OR create a new token group via TIU, if necessary:

        i. Click **Create** to display the **Create Token Group** dialog box.

        ii. Specify the ***Token Group Id**, ***Token Group Name**, **Token Vendor**. If necessary, specify the **Description** and **Segment** as well.

        > **Note:** Select "OATH" as the **Token Vendor** value.

        iii. Click **Submit**.

6. Select the default **Token Status** of tokens after import.

    > **Note:** The default value is "Activated".

7. [OPTIONAL] Tick the **Issuance Token Group for System Token Assignment** checkbox if you wish to use this feature.

    > **Note:** For more information, please refer to *UAS Administration Guide - Issuance Token Group*.

8. Specify the **Transport Key**.

    > **Note**: KCV and key count are read-only and auto-generated by the system.

9. Select the action to be taken in the event of a conflict. For cases where the token already exists in the system, the user can either overwrite existing tokens or terminate the token import activity.

10. [OPTIONAL] Enter the job description where the user can describe more about the current importing job.

11. [OPTIONAL] Click the **Preview Token** button to preview the tokens to be imported. Once clicked, a list of tokens with their details will be displayed.

12. Click the **Import Token** button to start the importing token process in the bulk job. Bulk job process is an asynchronous process that will be running in the background. A user need not wait

for the job to finish and once importing token process is started, the system will prompt a dialog box allowing the user to view the Bulk job detail.

Once the token import is successful, the tokens can now be used for any authentication or authorization purposes.

Import Token Activation Password (V-Key OATH only)

The token seeds must be imported before proceeding with this step. V-Key Token Activation PIN or password is imported in the same way as token import. The difference is to specify that the **Import Action** option is set to "Activation Password Import" and specify the PSKC file containing the activation passwords.

Configure Login Module

Navigate to **Administration Dashboard > Configuration > Authentication > Login Module** and select the pre-created login module: "DefaultOATHTokenLogin" or create a new login module and select the "OATH Token Login" implementation. Click the **Next** button and provide the details for the login module attributes before saving the changes.

Attributes	Description
*Login Module Id	Unique Id of login module.
*Login Module Name	Name of login module.
Description	Description of login module.
Version	Login module entry version.
Implementation	Type of login module implementation, i.e. "OATH Token Login".
Handler	Handler of the login module, i.e. "OATHTokenLogin".
Configuration Tab	
Login Configuration	
Login Policy	The login policy specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as

Attributes	Description
	other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect.
Authentication Level	Unused
Token Vendor	
Token Vendor	Refers to the OATH token vendor. Default: OATH
Retry Configuration	
Auto Retry	Setting to "true" will perform auto-retry for every token assigned to the user. **Note:** If set to "true": • Only "Response-Only (RO)" mode will be functional for this login module. • Bad login count (IThreshold) checking has to be disabled in the corresponding OATH Token Policy. Default: false

Configure Authentication Realm

Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and select the pre-created login module "DefaultOATHToken" or create a new authentication realm and provide the details for the authentication realm attributes before saving the changes.

Attributes	Description
*Authentication Realm Id	Unique Id of authentication realm.
*Authentication Realm Name	Name of authentication realm.
Description	Description of authentication realm.
Version	Authentication realm entry version.
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.

Attributes	Description
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in the user store(s) or other validation methods).
First(1st) Login Module	The first login method to use for user authentication.
Second(2nd) Login Module	Optional subsequent user authentication method.
Third(3rd) Login Module	Optional subsequent user authentication method.
Fourth(4th) Login Module	Optional subsequent user authentication method.
Fifth(5th) Login Module	Optional subsequent user authentication method.
Enable Login/Logout History Saving	Set this to "false" to increase performance if tracking of login/logout history is not required in your project or during a stress test. Default: true
User Response	On successful authentication, return name-value pair responses specified in this User Response.
Realm-Based Session Policy	The realm-based session policy of this authentication realm. If no realm-based session policy is assigned, then the session policy defined in Global Settings will be used.

An authentication realm can be configured to provide a chained authentication known as multiple-factor authentication (MFA) of up to five factors of authentication. It can either be configured as a 2-step or 1-step authentication depending on the login module configured. 2-step authentication is where a username and static password are sent for authentication first before prompting the user to enter the OATH OTP. Whereas, 1-step authentication is where password and OATH OTP are combined and submitted together during authentication.

The 2-step authentication can be set by creating an Authentication Realm where the **Second(2nd) Login Module** is set to use the "OATH Token Login" login module, e.g. "DefaultOATHTokenLogin". The **First(1st)Login Module** can be set to use "System Password Basic" or "Ldap Login".

On the other hand, the 1-step authentication can be set by creating an Authentication Realm where the **First(1st) Login Module** is set to use a **Composite Login** login module. This login module can support a combination of a dynamic password (a.k.a. OTP) and static password in the form of <OTP><Static Password> or <Static Password><OTP>.

Configure User Accounts

A user may belong to a default store or external user store. Refer to *AccessMatrix Common Administration Guide - Manage User Stores* to know about the different types of user stores and how to configure it.

To create a new user in the default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes that are necessary for OATH token implementation before saving the changes.

Attributes	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of user store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to the user. Click **Add** to display the **Choose Authentication Realm** dialog box. Select the authentication realm created in the previous section and click **OK**. Both the authentication realm and the login module(s) associated with it will be assigned to the user.	

Refer to *AccessMatrix Common Administration Guide - Manage Users* for detailed information about users.

Assign Token to User

To assign tokens:

1. Navigate to **Operation Dashboard > User & Segment > User**. Search and select the user to assign with a token.

   > **Note**: A new user can also be created if necessary. Refer to *AccessMatrix Common Administration Guide - Manage Users* for detailed information about users.

2. Click the **Token Assignment** button and select a **Token Reassignment** option:

   a. To allow a system to assign an available OATH token from the selected token group, select the **System Assigned** option and select the group from the **Token Group** drop-down list.

b. To manually assign a token to a user, select the **Enter Serial Number/ID** option and enter the OATH token serial number. Select "OATH" from the **Token Vendor** drop-down list.

c. Click the **OK** button to start the token assignment process. An information message for a successful token assignment will be displayed. Click **Yes** to continue token assignment, otherwise, click the **No** button.

d. The assigned token will be displayed under **User & Segment > User > Assigned Token** tab.

Token Administration

The administrator can carry out important tasks associated with tokens such as token assignment/unassignment, token resync, token status change, token sharing, and others. Refer to *UAS Administration Guide - Token Administration* for detailed steps and information about these tasks.

6. UAS OATH Token Implementation Using API

The following pages provide detailed information on the UAS OATH token APIs:

- Token Activation
- Login
- Token Verification
- Token Management
- Errors

6.1. Token Activation

OATH tokens can also be generated and managed directly via API. The API supports token generation, verification, assignment/unassignment, changing of token status, resyncing of tokens, and others.

Create and Assign OATH Token

- Use /token/OATH/create-assign-and-encrypt to create a new OATH token and assign it to a user.

  Note: This is useful for self-enrollment of a token by the user.
- The application needs to generate a key pair and send the public key to UAS. The public key will be used to encrypt the token seed generated by UAS for the token when sending it back.
- The token seed is further double-encrypted with a symmetric key derived from a generated activation password for the token. Thus, the application will also need to call /token/activationPassword/get to retrieve this when decrypting the token seed.
- The decrypted token seed will be securely stored in the user's token app.
- After the assignment of a token, the token has yet to be activated and will be in the "PreActivation" state.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Create and Assign OATH Token API Call

URL: /token/OATH/create-assign-and-encrypt

Request

JSON

```
{
  "sessionToken": "sessionToken of user with rights to
assign token",
  "userId": "demouser",
  "batchId": "token batch ID",
  "groupId": "token group ID",
  "clientPublicKey": "string",
  "clientPublicKeyId": "string",
  "tokenSpecs": [
  {
    "appId": "APP01",
    "authMode": "OTP",
    "algorithm": "urn:ietf:params:xml:ns:keyprov:pskc:totp",
    "responseFormatLength": "6",
```

Create and Assign OATH Token API Call

```
    "timeInterval": 30,
    "suite": "HmacSHA256"
  },
  {
    "appId": "APP02",
    "authMode": "CR",
    "algorithm": "urn:ietf:params:xml:ns:keyprov:pskc:ocra",
    "responseFormatLength": "6",
    "timeInterval": 30,
    "suite": "OCRA-1:HOTP-SHA256-6:QA4-T5S"
  },
  ],
  "params":{
        "serialNumberOrdered": true,
        "serialNumberPrefix": "string",
        "status": "string",
        "model": "string",
        "name": "string",
  }
}
```

Response

JSON


```
{
  "result": {
    "tokenInfoBean": {
        "serialNum": 12345678,
        "vendor": "OATH",
        "model": "OATH",
        "name": "AM5",
        "assignedUserUUIDs": [],
        "assignedUserIds": [],
        "UUID": "string",
        "version": 0
    },
    "specs": {
        "APP02": {
            "authMode": "CR",
            "algorithm": "o",
            "suite": "OCRA-1:HOTP-SHA256-6:QA4-T5S",
            "responseFormatLength": 0,
            "timeInterval": 5,
            "time": 0,
            "timeDrift": 0,
            "counter": 0,
            "version": 0
        },
        "APP01": {
            "authMode": "OTP",
            "algorithm": "t",
            "suite": "HmacSHA256",
            "responseFormatLength": 6,
            "timeInterval": 5,
```

Create and Assign OATH Token API Call

```
            "time": 0,
            "timeDrift": 0,
            "counter": 0,
            "version": 0
        }
    }
}
```

Create and Assign OATH Token (Google Authenticator)

- Use /token/OATH/create-and-assign/google-authenticator to create a new OATH token and assign it to a user.

  Note: This is useful for self-enrollment of a token by the user.
- After the assignment of a token, the token has yet to be activated and will be in the "PreActivation" state.
- The token seed generated by UAS will be returned.
- Application will generate a QR code for a Google key-URI format string using the token seed.
- User will scan the QR code with the Google Authenticator app to add the new token.
- User will generate a first OTP with the new token.
- Application calls UAS to verify OTP and activate the token if successfully verified. The token will then be in the "Activated" state.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Create and Assign OATH Token API Call

URL: /token/OATH/create-assign/google-authenticator

Request

JSON

```
{
  "sessionToken": "sessionToken of user with rights to
assign token",
  "userId": "demouser",
  "batchId": "token batch ID",
  "groupId": "token group ID",
  "tokenSpecs": [
```

Create and Assign OATH Token API Call

```
  {
    "appId": "APP01",
    "authMode": "OTP",
    "algorithm": "urn:ietf:params:xml:ns:keyprov:pskc:totp",
    "responseFormatLength": "6",
    "timeInterval": 30,
    "suite": "HmacSHA1"
  }
  ]
}
```

Response

JSON

```
{
  "result": {
    "tokenInfoBean": {
        "serialNum": 12345678,
        "vendor": "OATH",
        "model": "OATH",
        "assignedUserUUIDs": [],
        "assignedUserIds": [],
        "UUID": "string",
        "version": 0
    },
    "specs": {
        "APP01":
"A8D420220412112801302C02146B3B143679AD4151E9"
    }
}
```

Get Activation Password

- For the application or token app to get the token activation password from UAS.

REST Sample Code

Get Activation Password API

URL: /token/activationPassword/get

Request

JSON

```
{
  "sessionToken": "Session token with admin rights to
activate token",
```

Get Activation Password API

```
  "vendor": "OATH",
  "serialNumber": "GG0012345678",
}
```

Note:

- sessionToken: The session token of an admin user, also called an admin session.
  An admin session is one of various authentication methods for admins/application servers to prove that they have the necessary authorization to use certain AccessMatrix REST APIs. Other methods include:
  - Dynamic agent session
  - Access token

  Refer to *Using AccessMatrix REST APIs - Configuring Authentication Methods for Accessing AccessMatrix REST APIs* or more information on each of these methods.

Response

JSON

```
{
  "result":   "34564321"
}
```

Activate Token By OTP

- Use token/activateByOTP to activate a token with a first-time OTP generated by the token.
- The OTP will be verified by UAS.
- If successful, the token state will be changed from "PreActivation" to "Activated" and the token will be ready to use.

REST Sample Code

Activate Token by OTP

URL: /token/activateByOTP

Request

JSON

```
{
  "sessionToken": "Session token with admin rights to
activate token",
  "vendor": "OATH",
  "serialNumber": "GG0012345678",
```

Activate Token by OTP

```
  "OTP": "34564321"
}
```

Note:

- sessionToken: The session token of an admin user, also called an admin session.
  An admin session is one of various authentication methods for admins/application servers to prove that they have the necessary authorization to use certain AccessMatrix REST APIs. Other methods include:
    - Dynamic agent session
    - Access token

  Refer to *Using AccessMatrix REST APIs - Configuring Authentication Methods for Accessing AccessMatrix REST APIs* or more information on each of these methods.

Response

JSON


```
{
  "result":  {
  "authenticationCode": -1,
  "params": {
   "tokenSerNum": "GG0012345678",
   "tokenStatus": "Activated",
   "vendorId": "OATH",
   "otp": "34564321",
   "tokenUUID": "C_fw1KWNoI",
   "tokenModel": "OATH"
  }
}
```

6.2. Login

OATH Token (OTP & Challenge Response) Login Module

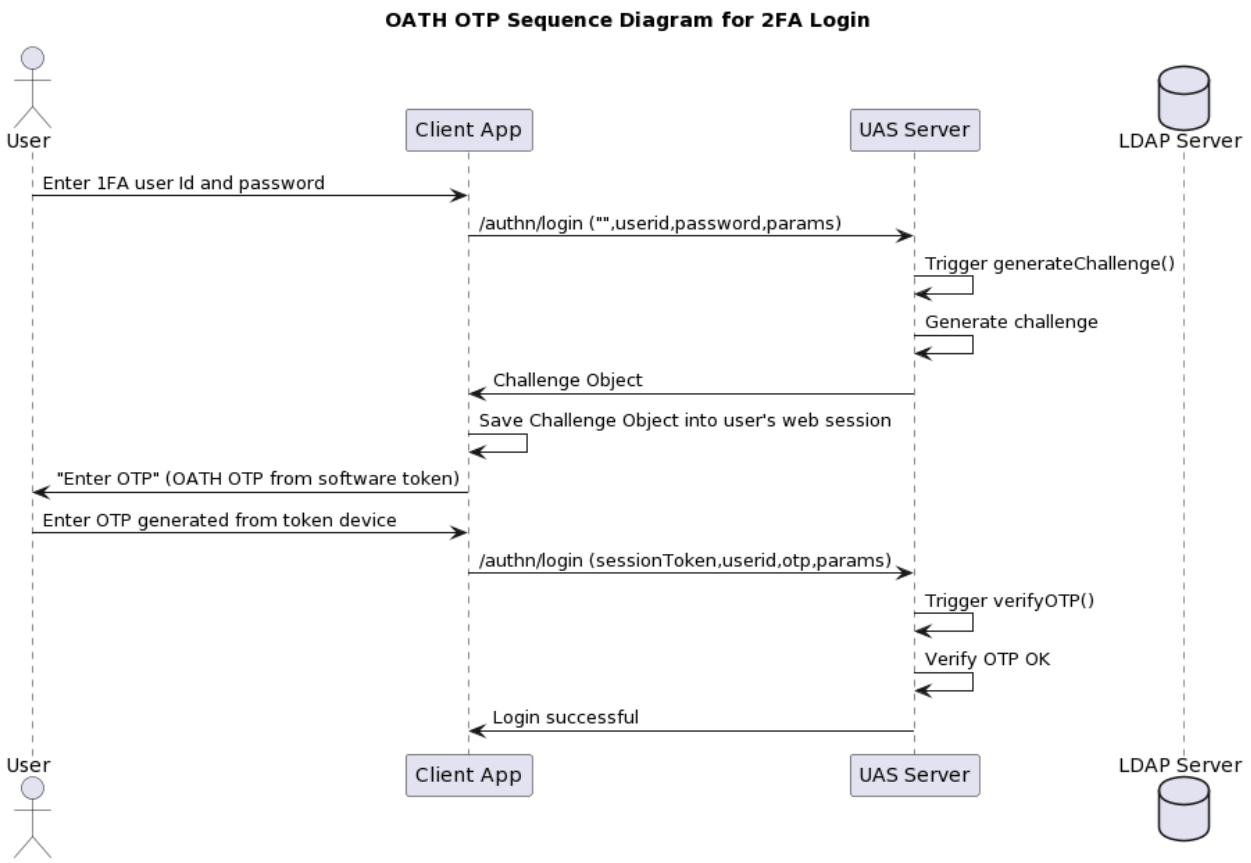
- Use /authn/login to log a user in.
- Refer to the *AccessMatrix REST API Specification* for more details.

Sequence Diagram

A. OATH OTP Sequence Diagram

This section covers the two-factor authentication use case of OATH token OTP through the REST AuthenticationService Login URL /authn/login. The sequence diagram below shows the basic flow of the API calls for the OATH token OTP login.

1. The REST API /authn/login (sessionToken, userId, password, params) is called.
    - The user passes in the authentication realm as realmId in params.
2. UAS generates a challenge if the authentication realm passed in as the parameter is defined for OATHTokenLogin.
3. The REST API /authn/login (sessionToken, userId, password, params) is invoked for the 2nd-factor login.
    - The OTP generated from the token device is used as the password.
    - The authentication realm is passed in through params.

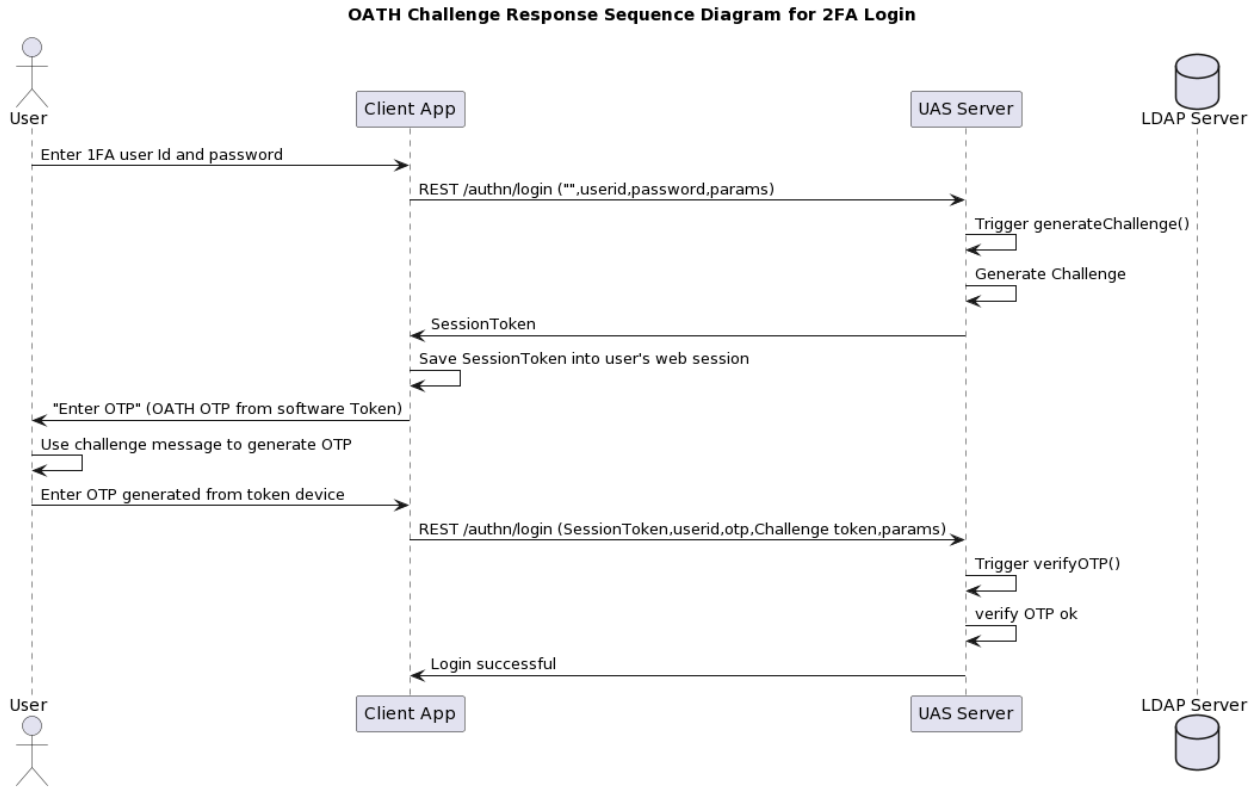


B. OATH Challenge Response Sequence Diagram

This section covers the two-factor authentication use case of OATH token Challenge Response through the REST AuthenticationService Login URL /authn/login. The sequence diagram below shows the basic flow of the API calls for the OATH token Challenge Response login.

1. The REST API /authn/login (sessionToken, userId, password, challenge, params) is called and UAS generates a challenge object.
    - The user passes in the authentication realm as realmId in params.
    - The user passes in authMode in the JSON object params to define the ChallengeResponse authMode ({"params":{"authMode":"CR"}}).
2. The sessionToken returned from the 1st login is stored for use in the 2nd login.
3. The challenge object is returned.
4. The challenge code is keyed into the token device to generate the response OTP.
5. The REST API /authn/login(sessionToken, userId, password, challenge token, params) is invoked for the 2nd-factor login.

- sessionToken is the sessionToken stored from the 1st login.
- password is be the OTP generated from the token device.
- challenge token is the challenge from /token/generateChallenge.
- The authentication realm is passed in through params.



REST Sample Code

A. OATH OTP REST API Sample

2FA Login API Call
First Login Call • "OATHOtp2FA" is a sample realm with 2FA comprised of the SystemPasswordBasic and OATH token login modules. • For 2FA (e.g. SystemPasswordBasic login module used with OATH token login module for the 2FA realm), the password argument is mandatory.   ◦ For 1FA (e.g. only OATH token login module used for 1FA realm), the password argument should be the OTP generated by OATH token. • The 1st login triggers generateChallenge() at the server-side to generate a ChallegeTO object.
URL: /authn/login

2FA Login API Call

Request

JSON

```
{
  "sessionToken": "",
  "userId": "otpuser",
  "password": "password for first login module",
  "realmId": "OATHOtp2FA"
}
```

Response

JSON

```
{
  "result": {
    ...
    "sessionToken": "sessionToken returned in first call",
    "challenge": {
      "code": "Please provide user id and password for login
module DefaultOATHTokenLogin",
              "authenticationCode": 18889,
                "message": "Authentication Factor #2
required",
                "params": {
                  "loginModule": "DefaultOATHTokenLogin",
                  "authnLevel": 1,
                  "msgArgs_0": [
                    "2",
                    "DefaultOATHTokenLogin"
                  ],
                  "msgLabel_0": "Exc_18889_2",
                  "loginMethod": "Form",
                  "challenge": "Authentication Factor #2
required",
                  "msgCount": 1,
                  "factor": 2
                }
    }
  }
}
```

Second Login Call

URL: /authn/login

Request

JSON

```
{
  "sessionToken": "sessionToken returned in first call",
  "realmId": "OATHOtp2FA",
```

2FA Login API Call

```
  "userId": "otpuser",
  "password": "OTP generated by OATH token"
}
```

Response (on successful login)

JSON

```
{
  "result": {
    "authenticated": true,
    "realmId": "OATHOtp2FA",
    "userId": "otpuser",
    "sessionToken": "authenticated sessionToken should be
saved for successive calls",
    "loginModuleStates": [
      {
        "authenticated": true,
        "canChangePassword": true,
        "id": "SystemPasswordBasic",
        "loginModuleHandlerClass": "SystemPasswordBasic"
      },
      {
        "authenticated": true,
        "canChangePassword": false,
        "id": "DefaultOATHTokenLogin",
        "loginModuleHandlerClass": "OATHTokenLogin"
      }
    ],
  }
}
```

B. OATH Challenge Response REST Sample

2FA Login API Calls

First Login Call

- 1st-FA login
  - Password argument is mandatory if the SystemPasswordBasic login module is used with the OATH token login module for 2FA realm
  - Specify "authMode" for the challenge response "CR"

URL: /authn/login

Request

JSON

```
{
  "sessionToken": "",
```

2FA Login API Calls

```
  "realmId": "OATHOtp2FA",
  "userId": "otpuser",
  "password": "password of first login module",
  "params": {
    "authMode": "CR"
  }
}
```

Response

JSON

```
{
  "result": {
    "sessionToken": "sessionToken returned in first call",
    "challenge": {
        "code": "Please provide user id and password for
login module DefaultOATHTokenLogin",
        "authenticationCode": 18889,
        "message": "Authentication Factor #2 required",
        "params": {
            "loginModule": "DefaultOATHTokenLogin",
            "authnLevel": 1,
            "msgArgs_0": [
                "2",
                "DefaultOATHTokenLogin"
            ],
            "msgLabel_0": "Exc_18889_2",
            "loginMethod": "Form",
            "challenge": "Authentication Factor #2
required",
            "msgCount": 1,
            "factor": 2
        }
    },
  }
}
```

Second LoginCall

- Key in the challenge message into the token device to generate a response.

URL: /authn/login

Request

JSON

```
{
  "sessionToken": "sessionToken returned in first call",
  "challengeToken": "297333",
  "realmId": "OATHOtp2FA",
  "userId": "otpuser",
  "password": "response generated by token device"
```

2FA Login API Calls

```
}
```

Response (on successful login)

JSON

```
{
  "result": {
    "sessionToken": "authenticated sessionToken should be
saved for successive calls",
    "authenticated": true,

  }
}
```

6.3. Token Verification

Verify OTP

- Use /token/user/assigned/list to retrieve the assigned tokens of a user based on the token vendor, token model, user store Id and user Id/UUID.
- Use /token/verifyOTP to verify the OTP.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Get Assigned Tokens by User and Token Model API Call

URL: /token/user/assigned/list

Request (by user ID)

JSON

```
{
  "sessionToken": "session with rights to list assigned
tokens",
  "userId": "demouser",
  "vendor": "OATH",
  "tokenModel": "YESsafe"
}
```

Request (by user UUID)

JSON

```
{
  "sessionToken": "session with rights to list assigned
tokens",
  "userUUID": "UT49mv1Wab",
  "vendor": "OATH",
  "tokenModel": "YESsafe"
}
```

Response

JSON

```
{
  "result": [
      {
      "serialNum": "011427969653",
      "vendor": "OATH",
      "model": "YESsafe",
```

Get Assigned Tokens by User and Token Model API Call


```
      "id": "OATH-011427969653",
      "UUID": "DTBzLzigMe",
      "version": 4,
      "status": "Activated"
      }
    ],
  "amProcessingTimeMillis": 13
}
```


Verify OTP By Serial Number API Call

URL: /token/verifyOTP

Request

JSON


```
{
  "sessionToken": "session with rights to verify OTP",
  "vendor": "OATH",
  "serialNumber": "011427969653",
  "OTP": "356137"
}
```


Response

JSON


```
{
  "result": {
    "authenticationCode": -1,
    "params": {
      "tokenSerNum": "011427969653",
      "tokenStatus": "Activated",
      "vendorId": "OATH",
      "otp": "356137",
      "tokenUUID": "ZIbXATiwB5",
      "tokenModel": "YESsafe"
    }
  },
  "amProcessingTimeMillis": 25
}
```


Generate Challenge

- Use /token/generateChallenge to generate a challenge for the user to get a token response.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Generate Challenge API Call

URL: /token/generateChallenge

Request

JSON

```
{
    "dynamicAgentSession": true,
    "vendor": "OATH",
    "serialNumber": "011427969653"
}
```

Response

JSON

```
{
    "result": {
        "challengeToken": "856139",
        "authenticationCode": -1,
        "params": {
            "tokenSerNum": "011427969653",
            "tokenStatus": "Activated",
            "challenge": "856139",
            "vendorId": "OATH",
            "tokenUUID": "4YLIFHzPQD",
            "tokenModel": "YESsafe"
        }
    },
    "amProcessingTimeMillis": 39
}
```

Verify Response

- Use /token/verifyResponse to verify the response for a given challenge.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Verify Response API Call

URL: /token/verifyResponse

Request

JSON

Verify Response API Call

```
{
    "dynamicAgentSession": "true",
    "serialNumber": "011427969653",
    "vendor": "OATH",
    "challenge": "856139",
    "response": "112233"
}
```

Response

JSON

```
{
    "result": {
        "authenticationCode": -1,
        "params": {
            "tokenSerNum": "011427969653",
            "tokenStatus": "Activated",
            "vendorId": "OATH",
            "tokenUUID": "4YLIFHzPQD",
            "tokenModel": "YESsafe"
        }
    },
    "amProcessingTimeMillis": 39
}
```

Verify Digital Signature

- Use /token/verifyDigitalSignature to verify the digital signature for a given input data.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Verify Digital Signature API Call (REST)

URL: /token/verifyDigitalSignature

Request

JSON

```
{
      "dynamicAgentSession" : "true",
      "serialNumber" : "4250326834",
      "vendor" : "OATH",
      "data" : [
        "32233223",
        ...
      ],
```

Verify Digital Signature API Call (REST)

```
      "signature" : "753560"
}
```

Response

JSON


```
{
  "result": {
    "authenticationCode": -1,
    "params": {
      "tokenModel": "YESsafe",
      "tokenSerNum": "4250326834"
      "tokenUUID": "4YLIFHzPQD",
      "tokenStatus": "Activated",
      "vendorId": "OATH",
    }
  },
  "amProcessingTimeMillis": 256
}
```

6.4. Token Management

Assign Token/ Unassign Token/ Reassign Token

- Use /token/assign to assign and/or unassign tokens to a holder (typically, a user or user group).

  Note: This is useful for self-enrollment of a token by the user.

- To assign a token, make sure the status of the token being assigned must be "Activated" before making a token assignment.
- After unassigning the token, it will be in the "Unassigned" state.
- Use /token/reassign to reassign a token to a holder; reassign is an action that combines the calling of unassign token + assign token.
- After reassigning a token, it will be in the "Activated" state.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Assign Token/ Unassign Token/ Reassign Token API Call

URL: /token/reassign

Request

JSON

```
{
  "sessionToken": "sessionToken of user with rights assign
token",
  "userId": "demouser",

  "vendorToUnassign": "OATH",
  "serialNumberToUnassign": "011427969653",

  "vendor": "OATH",
  "serialNumber": "880976741596"
}
```

Response

JSON

```
{
  "result": {
    "challenge": {
      "authenticationCode": -1,
      "params": {
```

Assign Token/ Unassign Token/ Reassign Token API Call


```
        "assignedTokenModels": [
          "OATH"
        ],
        "messages": [
          "Token ZIbXATiwB5 011427969653 has no more
assignment; status set to Unassigned"
        ],
        "unassignedTokenUUIDs": [
          "ZIbXATiwB5"
        ],
        "assignedTokenSerialNumbers": [
          "880976741596"
        ],
        "assignedTokenUUIDs": [
          "DTBzLzigMe"
        ]
      }
    },
    "responses": [
      {
        "assignedTokenModels": [
          "OATH"
        ],
        "messages": [
          "Token ZIbXATiwB5 011427969653 has no more
assignment; status set to Unassigned"
        ],
        "assignedTokenSerialNumbers": [
          "880976741596"
        ],
        "assignedTokenUUIDs": [
          "DTBzLzigMe"
        ]
      }
    ]
  },
  "amProcessingTimeMillis": 72
}
```


Query Token Information

- Use /token/find to search for tokens based on the given search criteria.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Query Token Information API Call
URL: /token/find

Query Token Information API Call

Request

JSON

```
{
  "sessionToken": "...",
  "vendor_eq": "OATH",
  "serialNum_eq": "880976741596",
  "model_eq": "YESsafe",
  "template": "properties"
}
```

Response

JSON


```
{
  "result": [
    {
      "serialNum": "880976741596",
      "vendor": "OATH",
      "name": "OATH 880976741596",
      "objectClass": "Token",
      "model": "YESsafe",
      "id": "OATH-880976741596",
      "UUID": "DTBzLzigMe",
      "version": 7,
      "properties": [
        {
          "otpLength": "6",
          "authMode": "OTP",
          "suite": "HmacSHA256",
          "badLoginCount": "11",
          "lastSuccAuth": "2023-10-27T03:51:51Z",
          "appId": "YESsafeApp",
          "timeInterval": "30",
          "locked": "false",
          "timeDrift": "-5",
          "lastTotp": "56612618",
          "manufacturer": "i-Sprint",
          "algorithm": "t"
        }
      ],
      "status": "Activated"
    }
  ],
  "amProcessingTimeMillis": 14
}
```


List Assigned Tokens of User

- Use /token/user/assigned/list to list the tokens assigned to a user.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

List Assigned Tokens API Call

URL: /token/user/assigned/list

Request

JSON

```
{
  "sessionToken": "...",
  "userId": "demouser",
  "vendor": "OATH",
  "tokenModel": "YESsafe"
}
```

Response

JSON

```
{
  "result": [
    {
      "serialNum": "880976741596",
      "vendor": "OATH",
      "model": "YESsafe",
      "id": "OATH-880976741596",
      "UUID": "DTBzLzigMe",
      "version": 7,
      "status": "Activated"
    }
  ],
  "amProcessingTimeMillis": 14
}
```

Change Token Status

- Use /token/changeStatus to change the token status.
    - By serial number: change the token status based on the token serial number.
    - By token UUID: change the token status based on the token UUID.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Change Token Status API Call

URL: /token/changeStatus

Change Token Status API Call

Request (by Token UUID)

JSON

```
{
  "sessionToken": "...",
  "UUID": "DTBzLzigMe",
  "status": "Activated"
}
```

Request (by Serial Number)

JSON

```
{
  "sessionToken": "...",
  "vendor": "OATH",
  "serialNumber": "880976741596",
  "status": "Activated"
}
```

Response

JSON

```
{
  "result": {
    "challenge": {
      "authenticationCode": -1,
      "params": {
        "tokenSerNum": "880976741596",
        "tokenStatus": "Activated",
        "vendorId": "OATH",
        "tokenUUID": "DTBzLzigMe",
        "tokenModel": "YESsafe",
        "oldTokenStatus": "Locked"
      }
    },
    "responses": [
      {}
  ]
    },
  "amProcessingTimeMillis": 39
```

Parameters & Exceptions

Exceptions

Attributes	Description
30001	The token is not found for token UUID.

Attributes	Description
49999	Other reasons: No more tokens available/current token status is not assignable.
20000	Authorization fail, login user does not have the token admin role.

Resync Token

- Use /token/resync to resync the token.
- OTP token internal clock may drift (for environmental reasons, a big temperature difference or after a period of very long activity) and become invalid in the allowed time window for OTP verification. The resync function needs to be called to erase all time synchronization information, thus forcing the next authentication to be considered as the first one.
- Application must call the UAS Resync Token API with various parameters, including token UUID, vendor Id, and token serial number. UAS will validate the token access Id and token serial number against its database. It may return different error codes for "wrong token serial number" and "re-sync failure". The request can be made by either:
    - Using serial number: vendor and serialNumber must be specified.
    - Using token UUID: UUID must be specified.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Resync Token API Call

URL: /token/resync

Request

JSON

```
{
  "sessionToken": "...",
  "UUID": "DTBzLzigMe"
}
```

Request (by token UUID)

JSON

```
{
  "sessionToken": "...",
```

Resync Token API Call

```
  "vendor": "OATH",
  "serialNumber": "880976741596"
}
```

Response

JSON

```
{
  "result": {
    "challenge": {
      "authenticationCode": -1,
      "params": {
        "tokenSerNum": "880976741596",
        "tokenStatus": "Activated",
        "vendorId": "OATH",
        "tokenUUID": "DTBzLzigMe",
        "tokenModel": "YESsafe",
        "TokenMgtIsVerifyingAfterResync": false
      }
    },
    "responses": [
      {}
  ]
    },
  "amProcessingTimeMillis": 19
}
```

Parameters & Exceptions

Additional optional parameters for the JSON object params:

JSON

```
{
  "params": {
    "appId": "...",
    "newStatus": "...",
  }
}
```

Parameter	Description	Supported Values
appId	Token blob to resync	
newStatus	New token status after resync is done	

Exceptions:

Error Code	Description
30001	Token is not found for token UUID.
20000	Authorization fail, login user does not have the token admin role.

6.5. Errors

AccessMatrix REST API Error Codes, Faults or Exceptions

Authentication Faults and Error Codes

Refer to Error Codes, Faults or Exceptions - "List of Authentication Faults/Error Codes" in the *AccessMatrix REST API Specification*.

Authorization Faults and Error Codes

Refer to Error Codes, Faults or Exceptions - "List of Authorization Faults/Error Codes" in the *AccessMatrix REST API Specification*.

Query Faults and Error Codes

Refer to Error Codes, Faults or Exceptions - "List of Query Faults/Error Codes" in the *AccessMatrix REST API Specification*.

System (Runtime) Faults and Error Codes

Refer to Error Codes, Faults or Exceptions - "List of System Faults/Error Codes" in the *AccessMatrix REST API Specification*.

7. UAS OATH Token Housekeeping Policy

The housekeeping policy contains rules on the housekeeping of the OATH token data in the database. To view the housekeeping policy, navigate to **Administration Dashboard > System > Housekeeping** and click **Housekeeping Policy** on the left pane. Select the **UAS** tab and click the **Edit** button. Save the changes after modification.

Below is the list of OATH token-related housekeeping attributes:

Attributes	Description
UAS Tab	
Tokens	
Purge non-SMS token if it is in a certain status for longer than the specified period	Specifies how long non-SMS tokens can remain in a certain status before they are purged. Define the status for the token, i.e. "Battery Expired", and its period in months, i.e. "24", and then add the status and its period to the list. **Note:** Period (Months) should be between 1 and 1200
Change non-SMS token status if it is not used for longer than the specified period	Specifies how long non-SMS tokens can remain unused before their current status is changed to a new status. Define the affected token group, the new status to change to, i.e. "Deactivated", and the time period for the tokens to remain unused before their status is changed to the new status. This attribute only affects tokens which meet the following requirements: • Token must be assigned to a user. • The current token status must have been defined as operational or usable. By default, this would be "Activated" and "Preactivation". The list of statuses can be configured using the attribute **Token Status for usable functions like verify OTP/signature/response etc.** For more information on how to configure this, refer to *UAS Administration Guide - Global Setting - Token Management*. Note: Period (months) should be between 1 and 1200.

8. Appendices

FAQs – How to protect OATH Token Seeds

Token file format

V-Key OATH Token seeds are protected using the PSKC (Portable Symmetric Key Container) XML file format.

Reference: https://datatracker.ietf.org/doc/html/rfc6030

Token seed confidentiality in transit

A transport key is used to protect token seeds in transit. This transport key is mailed separately from the PSKC file to the token administrator. The customer can request for the transport key to be separated into components which can each be mailed to different token administrators.

Token seed confidentiality during import

During the token import, token administrator(s) need to input the transport key component(s) to decrypt the token seeds in the PSKC file which are used for import.

YESsafe Tokens

Unlike V-Key tokens, YESsafe tokens are not imported. Instead, the token seed is generated when the token created and assigned to a user.

The token seed is generated by AccessMatrix UAS and then double encrypted with:

1. RSA with OAEP Padding using the token's client public key.
2. PBKDF2-derived AES key using the token's activation password.

Google Authenticator

Similar to YESsafe tokens, these tokens are not imported but created and assigned on demand.

Currently, Google Authenticator expects a clear base-32 encoded secret seed in the Google Key URI Format QR Code. Due to this, the transfer of the token seed to the token is not encrypted.

Token seed confidentiality in storage and in operation

After being imported or created, all OATH tokens will have a token BLOB containing the token seed and other security information that is encrypted and stored in the database.

The token BLOB (which includes the token seed) is encrypted with a token encryption key. A default token encryption key is generated by AccessMatrix UAS. An administrator can also generate new token encryption keys as required. These are typically AES keys.

During the token policy creation, you can select the specific token encryption key to be used for protecting the token BLOB.

Summary of keys used

Key Name	Cryptographic Algorithm	Purpose	Management
Transport Key	3DES or AES (as specified in PSKC specifications, AES is preferred)	Protect the token seeds contained in PSKC files during transit	Sent by V-Key to the customer for PSKC import (specific details must be requested from V-Key)
Token Encryption Key (TEK)	AES	Protect the token BLOB for storage	One default key is generated by AccessMatrix. New keys may be generated as required
Domain Master Key (DMK)	AES	Protect TEK and other encryption key	Derived using PBKDF2 with proprietary secret